



An Explainable Artificial Intelligence Based Conceptual Model for Financial Auditing to Enhance Audit Transparency and Quality

Supervisor: Dr. Gamal Kassem

Presented by: George Samy Nabih

ID: 58-21333

Submission date: 17-May-2026

Abstract

Financial auditing faces growing challenges as data volumes increase and traditional methods struggle to keep up. While artificial intelligence has shown strong results in fraud detection and anomaly identification, most AI systems used in auditing operate as black boxes, making their outputs difficult to justify. This study addresses that problem by developing a conceptual model that integrates Explainable Artificial Intelligence into the financial auditing process to improve transparency and audit quality.

Following Design Science Research methodology, a structured literature review identified the scientific gap and guided the design of a twenty-seven-step pipeline covering data ingestion, anomaly detection, SHAP-based explanation, ERP control attribution, ISA standard retrieval, and narrative generation. A focused prototype covering steps 19 to 25 was built using Python, LangGraph, ChromaDB, and a locally hosted Llama 3.1 8B model, then evaluated through verification tests, validation tests, a retrieval accuracy analysis, and expert interviews with four domain professionals using the Delphi method.

The results confirm that the proposed system can produce transparent, standards-grounded audit findings that are traceable, modular, and aligned with professional auditing requirements.

Keywords: Explainable Artificial Intelligence, Financial Auditing, Design Science Research, Anomaly Detection, SHAP, Large Language Models, ISA Standards

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Dr. Gamal Kassem, for his guidance, constructive feedback, and continued support throughout the development of this thesis. His expertise and encouragement were instrumental in shaping the direction and quality of this work.

I am also grateful to the four domain experts who generously contributed their time and professional insight during the expert interview phase: the AI Engineer, the Full-Stack Software Engineer, the External Auditor, and the Internal Auditor. Their evaluations significantly improved the conceptual model and the demonstration prototype.

Finally, I would like to thank my family and friends for their patience and encouragement throughout this journey.

George Samy Nabih

May-2026

Table of Contents

Abstract.....	ii
Acknowledgement.....	iii
List of Abbreviations	vii
List of Figures.....	ix
List of Tables.....	x
List of Equations	xi
1 Introduction.....	1
1.1 Motivation.....	1
1.2 Problem Definition.....	2
1.3 Research Objective	2
1.4 Methods Used	2
1.5 Structure of Work.....	3
2 Foundations of Financial Auditing.....	4
2.1 Definition and Objectives of Financial Auditing.....	4
2.2 Core Auditing Types	4
2.2.1 Internal	4
2.2.2 External.....	5
2.2.3 Forensic.....	5
2.3 Auditing Process and Procedures.....	5
2.4 Auditing Quality.....	6
2.5 Limitations and Challenges in Traditional Auditing.....	7
3 Artificial Intelligence Main Terminologies.....	8
3.1 Definition of Artificial Intelligence.....	8
3.2 Major Domains of Artificial Intelligence.....	8
3.2.1 Machine Learning.....	8
3.2.2 Deep Learning.....	9
3.2.3 Natural Language Processing.....	9
3.2.4 Explainable Artificial Intelligence	9

3.2.5 Generative Artificial Intelligence.....	10
3.3 Advantages and Disadvantages of Artificial Intelligence	11
4 AI-Driven Auditing.....	13
4.1 Generative AI for Auditing	13
4.1.1 Large Language Models in Auditing Activities.....	13
4.2 Supervised Techniques used in Auditing Activities.....	13
4.2.1 Support Vector Machines.....	13
4.2.2 Decision Trees.....	15
4.3 Unsupervised Techniques used in Auditing Activities.....	16
4.3.1 K-Means Clustering.....	16
4.3.2 Unsupervised NLP Anomaly Detection.....	17
4.4 Deep Learning used in Auditing Activity	17
4.4.1 Autoencoder Neural Networks.....	17
5 Literature Review	19
5.1 State of the Art	19
5.2 Discussion.....	26
5.3 Scientific Gap.....	32
6 Scientific Methodology	33
6.1 Design Science.....	33
6.2 Data Science.....	37
6.3 Quantitative and Qualitative	38
6.4 Literature Review.....	39
6.5 Solution Approach.....	40
7 Implementation	42
7.1 Problem identification.....	42
7.2 Design	43
7.2.1 Analysis.....	43
7.2.2 Modelling.....	47
7.3 Evaluation	61
7.3.1 Demonstration Introduction.....	62

7.3.2 Technical Stack and Dependencies	63
7.3.3 Installation Process	65
7.3.4 Demonstration Mapping	66
7.3.5 Verification and Validation.....	77
7.3.6 Expert Interview.....	83
7.3.7 Refinement.....	87
8 Conclusion	91
8.1 Summary.....	91
8.2 Limitations	93
8.2.1 Environmental Limitations	93
8.2.2 Conceptual Limitations.....	93
8.3 Future Work	94
8.4 Related Work.....	94
References.....	96
Statement of Autonomy.....	100

List of Abbreviations

AI: Artificial Intelligence
ALT: Alternative (UML sequence diagram notation)
API: Application Programming Interface
BERT: Bidirectional Encoder Representations from Transformers
CART: Classification and Regression Tree
CPU: Central Processing Unit
CRISP-DM: Cross-Industry Standard Process for Data Mining
CSRC: China Securities Regulatory Commission
CSV: Comma-Separated Values
CUDA: Compute Unified Device Architecture
DL: Deep Learning
DSR: Design Science Research
DSRM: Design Science Research Methodology
ERP: Enterprise Resource Planning
FMCG: Fast-Moving Consumer Goods
FN: False Negative
FP: False Positive
Gen AI: Generative Artificial Intelligence
GL: General Ledger
GNN: Graph Neural Network
GPU: Graphics Processing Unit
HCLA: Human-Centered LLM-Agent System
HTTP: Hypertext Transfer Protocol
IAASB: International Auditing and Assurance Standards Board
IIA: Institute of Internal Auditors
ISA: International Standards on Auditing
ISO: International Organization for Standardization
IT: Information Technology
JSON: JavaScript Object Notation
KDD: Knowledge Discovery in Databases
KNN: K-Nearest Neighbor
LLM: Large Language Model
LOF: Local Outlier Factor
ML: Machine Learning
MSE: Mean Squared Error
NER: Named Entity Recognition
NLG: Natural Language Generation

NLP: Natural Language Processing
NLU: Natural Language Understanding
PDF: Portable Document Format
PNG: Portable Network Graphics
PRISMA: Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PyPI: Python Package Index
RAG: Retrieval-Augmented Generation
RAM: Random Access Memory
ROA: Return on Assets
SDK: Software Development Kit
SHAP: SHapley Additive exPlanations
SLR: Systematic Literature Review
SMEs: Small and Medium-sized Enterprises
SoD: Segregation of Duties
SQL: Structured Query Language
SVM: Support Vector Machine
TLAE: Tri-Layer Anomaly Explanation
TN: True Negative
TOE: Technology, Organization, Environment
TP: True Positive
UI: User Interface
UML: Unified Modeling Language
VRAM: Video Random Access Memory
WCSS: Within-Cluster Sum of Squares
XAI: Explainable Artificial Intelligence
XGBoost: Extreme Gradient Boosting
XML: Extensible Markup Language

List of Figures

Figure 1 Auditing Process.....	6
Figure 2 Machine Learning Subfields.....	9
Figure 3 LLM Overview.....	11
Figure 4 Confusion Matrix.....	14
Figure 5 Architecture and Mathematical Representation of the Autoencoder Neural Network ...	18
Figure 6 PRISMA Approach.....	21
Figure 7 Concept Map of AI Opportunities in Auditing.....	24
Figure 8 Information Systems Research Framework.....	33
Figure 9 DSRM Process Model	35
Figure 10 Proposed research process from Offermann.....	36
Figure 11 Phases of CRISP-DM Process Model for Data Mining.....	37
Figure 12 KDD Process	38
Figure 13 Three Round Delphi Process	39
Figure 14 Solution Approach Diagram.....	41
Figure 15 Conceptual Model Class Diagram.....	51
Figure 16 Conceptual Model Sequence Diagram	56
Figure 17 Chunking Process Code.....	67
Figure 18 Embedding and Indexing Process Code	69
Figure 19 Demonstration Components Code.....	70
Figure 20 LangGraph Code.....	71
Figure 21 UI Inputs.....	72
Figure 22 Retrieval Code.....	73
Figure 23 LLM Generation Prompt Code.....	75
Figure 24 Demonstration UI-1.....	76
Figure 25 Demonstration UI-2.....	77
Figure 26 SHAP Feature Retrieval Confusion Matrix.....	81
Figure 27 ERP Attribution Retrieval Confusion Matrix	82
Figure 28 Sequence Diagram Refinement	88
Figure 29 Demonstration Refinement Code	89
Figure 30 UI after Refinement.....	90

List of Tables

Table 1 Summary of Empirical Studies on AI in Financial Auditing	31
Table 2 Design-Science Research Guidelines.....	34
Table 3 Functional Requirements	49
Table 4 Non-Functional Requirements	50
Table 5 Verification Tests.....	79
Table 6 Validation Tests	80

List of Equations

Equation 1 Accuracy using Confusion Matrix Variables	14
Equation 2 Entropy	15
Equation 3 Information Gain	15
Equation 4 Gini Index.....	15
Equation 5 Euclidean distance.....	16
Equation 6 Mean.....	16
Equation 7 within-cluster sum of squares (WCSS)	16
Equation 8 Mean Squared Error (MSE).....	18

1 Introduction

Artificial intelligence (AI) is becoming an important tool in many business fields, including financial auditing. Today, auditors deal with exceptionally large and complex financial data and face strong pressure to detect fraud and ensure high audit quality. Traditional audit methods based on sampling and manual testing are no longer enough to meet these demands. AI techniques offer new ways to support auditors by improving risk analysis, fraud detection, and audit efficiency. However, for AI to be trusted and used effectively in auditing, its output must be transparent and explainable. This study addresses that requirement by developing a conceptual model that integrates Explainable Artificial Intelligence into the financial auditing process to enhance audit transparency and quality.

1.1 Motivation

The increasing volume and complexity of financial data in modern organizations have made traditional auditing methods insufficient. Auditors are expected to examine large datasets, identify hidden risks, detect fraud, and deliver reliable opinions within limited timeframes. Conventional audit approaches rely on manual sampling and rule-based procedures, which cannot handle the scale and variety of data produced in today's digital business environments. As a result, many audit tasks are slow, error-prone, and unable to capture patterns that exist across large and interconnected datasets.

Artificial intelligence offers a practical solution to these challenges. Techniques such as machine learning, deep learning, natural language processing, and generative AI have demonstrated strong capabilities in fraud detection, anomaly identification, risk assessment, and audit report generation. Several of these techniques, including Support Vector Machines, Decision Trees, K-Means clustering, Autoencoders, and Large Language Models, are already being studied and applied in auditing contexts. However, despite their technical performance, most of these systems produce outputs that auditors cannot easily interpret or justify. This creates strong motivation to study how AI can be integrated into auditing in a way that is not only accurate but also transparent, understandable, and aligned with professional standards.

1.2 Problem Definition

Although AI techniques have shown strong results in improving fraud detection and audit efficiency, most AI models used in auditing operate as black boxes. This means that their internal reasoning is not visible, and auditors cannot clearly understand why a specific transaction was flagged or how a particular conclusion was reached. In auditing, this is a serious problem because auditors are professionally and legally required to justify their findings and support their opinions with traceable evidence.

A structured review of recent academic literature confirms that this problem remains unresolved. Existing studies focus primarily on improving the technical accuracy of AI models without addressing how their outputs can be explained or connected to recognized professional standards. There is also a clear absence of practical artifacts that demonstrate how Explainable AI can be incorporated into the audit process in a structured and reliable way. This gap between AI capabilities and practical audit requirements defines the central problem that this study addresses.

1.3 Research Objective

The main objective of this research is to **develop a conceptual model** for integrating Explainable Artificial Intelligence (XAI) into the financial auditing process, addressing the identified scientific gap, through which audit transparency, interpretability, and reliability can be enhanced to improve the overall quality of AI-supported audit processes.

1.4 Methods Used

This study adopts Design Science Research as the primary research paradigm, following the structured process model proposed by Offermann et al. (2009), which organizes the research into three phases: problem identification, design, and evaluation.

In the problem identification phase, a structured literature review was conducted using academic sources published between 2021 and 2025, collected through Google Scholar and ResearchGate. Eleven candidate papers were identified and filtered to five peer-reviewed publications based on relevance and depth. From these papers, the scientific gap and research question was derived then the research objective was formulated.

In the design phase, existing partial solutions were analyzed to extract functional requirements, non-functional requirements, system components, and building blocks. Based on this analysis, a conceptual model was constructed and described from two perspectives: a structural perspective using a UML class diagram, and a behavioral perspective using a sequence diagram representing a twenty-seven-step end-to-end pipeline.

In the evaluation phase, a focused prototype was implemented covering steps 19 to 25 of the conceptual pipeline, which represents the most technically novel contribution of the work. The prototype was built using Python, LangGraph, ChromaDB, the BAAI/bge-large-en-v1.5 embedding model, and a locally hosted Llama 3.1 8B language model served via Ollama. The artifact was evaluated through structured verification and validation tests, followed by iterative expert interviews with four domain professionals, comprising two auditing practitioners and two software engineers, conducted using the Delphi method and structured around the TOE framework.

1.5 Structure of Work

This paper consists of eight chapters; after the Introduction, Chapter 2 explains the foundations of financial auditing, including the definition and objectives of auditing, the three core audit types, the structured audit process, audit quality, and the main limitations of traditional auditing in the big data environment. Chapter 3 presents the main concepts of artificial intelligence relevant to this study, including machine learning, deep learning, natural language processing, explainable artificial intelligence, and large language models, along with the advantages and disadvantages of AI. Chapter 4 discusses how different AI techniques are applied specifically in auditing activities, covering large language models, supervised methods such as Support Vector Machines and Decision Trees, unsupervised methods such as K-Means clustering and NLP-based anomaly detection, and deep learning methods such as Autoencoders. Chapter 5 provides a systematic literature review of five peer-reviewed papers published between 2021 and 2025, followed by a comparative discussion and the identification of the scientific gap. Chapter 6 presents the scientific methodology adopted for this study. Chapter 7 presents the full design, implementation, and evaluation of the proposed system, including the conceptual model, the focused prototype, verification and validation results, expert interviews, and refinements. The paper ends with a conclusion in Chapter 8, followed by the full list of references.

2 Foundations of Financial Auditing

To understand the auditing field, it is important to start with its basic meaning and purpose. Before explaining audit types and processes, the definition and main objectives of financial auditing must be clear. This chapter presents the definition of financial auditing and its key objectives.

2.1 Definition and Objectives of Financial Auditing

Auditing is the systematic and independent examination of financial records, accounts, and related documents to determine whether they present a true, fair, complete, and reliable view of an organization's financial position and transactions (Sharma & Panigrahi, 2013).

The main objective of auditing in modern organizations is to provide assurance that systems, performance measures, and internal controls operate reliably and transparently. Power (2000) argues that auditing has expanded beyond financial checks to become a wider mechanism for accountability, driven by rising public expectations for transparency and responsible governance. In this sense, audits aim to verify whether organizations are meeting their stated objectives and whether performance indicators are designed and reported in an auditable way. Another objective is to support regulatory compliance, as new public management reforms and quality assurance models require organizations to demonstrate that they monitor and control their own activities effectively. Auditing therefore becomes a tool for “control of control,” ensuring that internal processes are functioning as required. Overall, the purpose is not only to detect problems, but also to shape organizational behavior by encouraging consistent documentation, structured evaluation, and measurable performance (Power, 2000).

2.2 Core Auditing Types

After the definition and objectives of financial auditing are explained, different audit types are presented in this section, as they are used to serve different purposes.

2.2.1 Internal

Internal auditing is an independent, objective assurance and consulting activity designed to add value and improve an organization's operations. It helps an organization accomplish its objectives by bringing a systematic, disciplined approach to evaluate and improve the effectiveness of risk management, control, and governance processes (Ramamoorti, 2003).

2.2.2 External

External auditing is an independent assessment of an organization's financial information performed for stakeholders outside the entity. It is carried out by qualified auditors who are fully independent of the auditee and who report their conclusions directly to external parties, these audits follow requirements established by or for the users of the audit results. Compliance audits conducted by government agencies also fall under this category. Overall, external auditing provides essential assurance and accountability for parties who rely on accurate and objective financial information (Nieschwietz et al., 2000).

2.2.3 Forensic

A forensic audit is a specialized investigative engagement that applies forensic accounting techniques to identify, interpret, and communicate evidence related to underlying economic events and potential financial misconduct. Unlike traditional audits or single-event fraud examinations, a forensic audit is proactive, continuous, and principles-based, focusing on uncovering economic reality and detecting hidden or unstructured financial activities. It integrates accounting, criminology, digital forensics, and investigative auditing to evaluate risks, detect anomalies, and reveal financial malfeasance within an organization (Smith & Crumbley, 2009)

2.3 Auditing Process and Procedures

Auditing follows a clear and organized set of procedures to ensure accuracy and reliability. This section explains the main stages of the auditing process and its procedures.

Figure 1 shows the main stages of an audit program according to international regulatory standards such as ISO 9001 and ISO 19011. Regulatory standards are official rules and guidelines that organizations and auditors must follow to ensure quality, consistency, and compliance. The audit program begins with planning, where the purpose, objectives, risks, scope, resources, and timing of the audit are defined. A key step in this stage is risk assessment, which means identifying and evaluating possible problems that may affect the organization or the audit results. The second stage is implementation, which includes defining audit methods, selecting the audit team, and managing audit activities. This is followed by monitoring where audit performance is measured. Finally, the program ends with review and improvement, where management evaluates results and applies lessons learned to improve future audits. This structured process helps organizations maintain compliance and improve overall audit quality (Chiromo & Msibi, 2023).

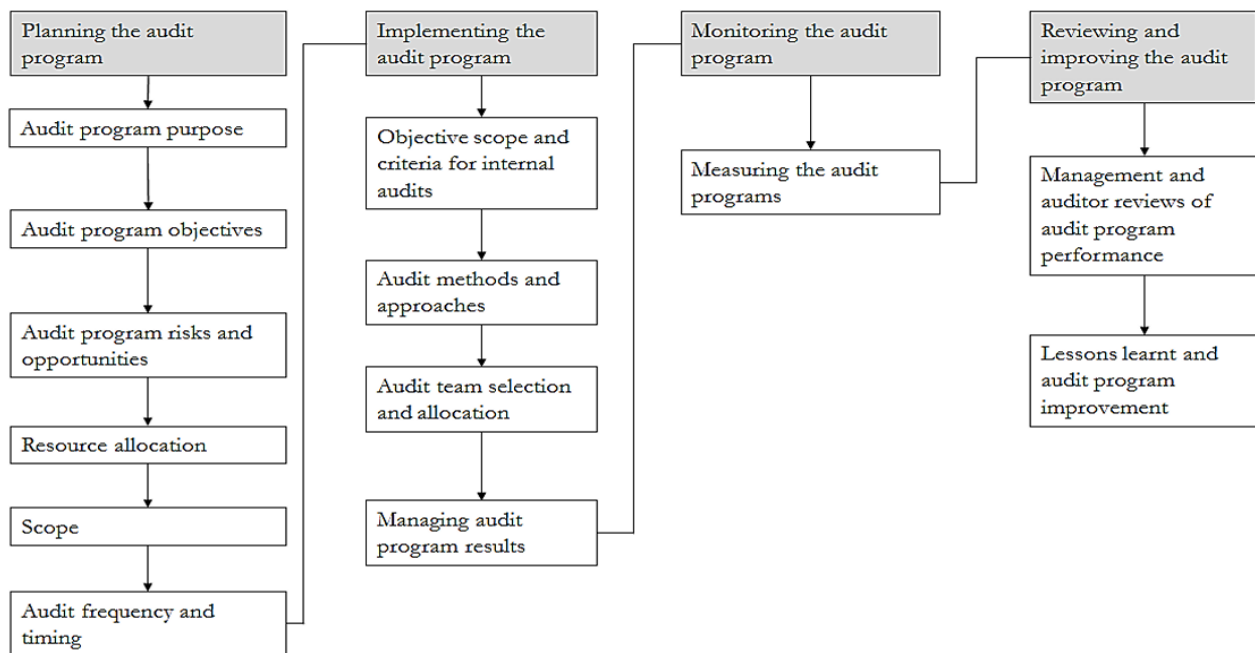


Figure 1 Auditing Process

(Chiromo & Msibi, 2023)

2.4 Auditing Quality

After explaining the auditing process and procedures, it is important to understand how the quality of auditing is evaluated. Audit quality shows how reliable and effective the audit work is. This section explains the meaning and importance of auditing quality.

Audit quality is the market's judgment of the probability that an auditor will discover errors in a client's accounting system and report these errors correctly. This definition shows that audit quality depends on two main factors: auditor competence and auditor independence. Auditor competence refers to the auditor's knowledge and skills that help in finding mistakes in financial statements. Auditor independence refers to the auditor's ability to report errors honestly without being influenced by the client. Both competence and independence are important and positive for audit quality, meaning that higher levels of each lead to better audit quality. The definition also assumes that these two factors are independent from each other, although in practice they may be related. Therefore, audit quality can be understood as the result of both strong technical skills and strong ethical objectivity in the audit process (DeAngelo, 1981).

2.5 Limitations and Challenges in Traditional Auditing

After understanding what audit quality means and why it is important, it is necessary to look at the problems that traditional auditing still faces. These challenges affect the efficiency and accuracy of audit work. This section explains the main limitations and challenges in traditional auditing.

Traditional auditing faces several challenges in the big data environment. **Big data** means large, fast, and varied data that comes from many sources, such as internal systems, customers, suppliers, e-commerce platforms, and social media. First, traditional audits use internal financial data from systems such as ERP and do not capture external and unstructured data. As a result, auditors do not see the full picture of a company's activities, which increases the chance of hidden risks and errors. Second, traditional audit tools are built for small, structured datasets. When data volume grows, these tools become slow and inefficient, so auditors cannot work in real time and may react too late to financial problems. Third, traditional audits still depend on sampling. In a big data setting, where data are complex and highly connected, small samples may not represent the full population, leading to lower accuracy and higher audit risk. Fourth, the technologies used are limited: common audit software can only perform basic checks and cannot process unstructured data like text, images, or videos, nor discover deeper relationships between different data sources. There is also a skills gap. Many auditors are trained only in accounting and auditing standards, but big data auditing requires knowledge of data mining, programming (e.g., Python, SQL), and machine learning, which many do not have. Finally, risk control becomes more difficult because digital data can be changed, lost, or attacked by hackers. Traditional methods are weak in protecting data security and in checking the authenticity of electronic audit evidence, which is easy to modify without clear traces. These problems reduce the reliability of audit conclusions and increase overall audit risk in the big data era (Zhao, 2025).

Because of the many challenges faced by traditional auditing in the big data environment, new technological solutions are required to improve audit performance. One of the most important solutions is artificial intelligence. To understand how AI can support auditing, it is first necessary to explain its basic concepts and main terms. Therefore, the next chapter introduces the main terminologies of artificial intelligence.

3 Artificial Intelligence Main Terminologies

To understand artificial intelligence in a clear way, it is necessary to know what it means. A simple definition helps explain the idea of AI and how it works. This chapter presents the definition of artificial intelligence and its subfields.

3.1 Definition of Artificial Intelligence

AI is understood as the study and development of computer systems capable of exhibiting behaviors associated with human intelligence, such as learning, reasoning, adapting, and self-correction. Also, the modern definitions emphasize “imitating intelligent human behavior”, and that most characterizations of artificially intelligent systems fall into four core categories: systems that think like humans, systems that act like humans, systems that think rationally and systems that act rationally (Sheikh et al., 2023, pp. 15–20).

3.2 Major Domains of Artificial Intelligence

After artificial intelligence is defined, its main parts are explained to show how it is applied in real systems.

3.2.1 Machine Learning

According to Janiesch et al. (2021) Machine learning is a part of AI where a computer program improves its performance with experience on a certain task. Instead of manually coding all the rules, ML automatically builds analytical models by learning from problem-specific training data. It finds patterns and relationships in data (e.g., classification, regression, clustering) and is widely used for things like fraud detection, credit scoring, forecasting, image, and speech recognition (Janiesch et al., 2021).

3.2.2 Deep Learning

As shown in Figure 2 Deep Learning is a subset of machine learning based on artificial neural networks. It uses deep neural networks (many layers, often with advanced units like convolutions) that can learn features automatically from raw, high-dimensional data (text, images, video, audio, time series). Therefore, DL often outperforms “shallow” ML algorithms on data, but usually needs more data, more computing power, and often behaves like a black box (Janiesch et al., 2021).

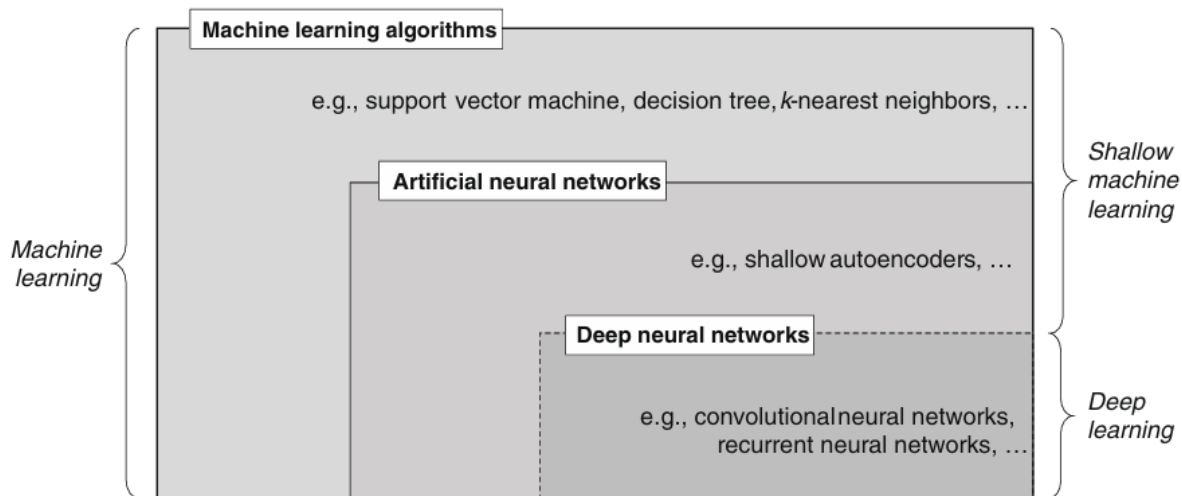


Figure 2 Machine Learning Subfields

(Janiesch et al., 2021)

3.2.3 Natural Language Processing

According to Jain, Kulkarni, and Shah, Natural Language Processing (NLP) is a field at the intersection of computer science, machine learning, and computational linguistics that focuses on making human–computer interaction easier and more efficient by enabling machines to learn the syntax and meaning of human language, process inputs such as text or speech, and produce meaningful outputs like written or spoken responses (Jain et al., 2018).

3.2.4 Explainable Artificial Intelligence

Explainable Artificial Intelligence (XAI) refers to methods that make AI systems understandable by providing insight into how and why a model produces its outputs, ensuring that its reasoning is accessible to humans. Transparency, as defined in the same framework, refers to the extent to

which the internal structure, computations, and decision-making mechanisms of a model can be directly examined and understood by humans. Within this definition, transparency includes the levels of simulatability, decomposability, and algorithmic transparency, which outline how accessible a model's internal logic is to a human observer. Together, XAI and transparency aim to support trust, accountability, and responsible use by enabling clear understanding of AI behavior. (Arrieta et al., 2020)

3.2.5 Generative Artificial Intelligence

Generative artificial intelligence (GenAI) is a type of artificial intelligence that uses a large dataset to create new content that is similar to the data it was trained on. It can generate many kinds of outputs, such as texts, audio, images, videos, objects, and data by learning patterns and relationships in huge datasets and then producing new examples in the same style (Håkansson & Phillips-Wren, 2024).

One important type of Generative AI is the Large Language Model, which is used only for language tasks.

3.2.5.1 Large Language Models

Large Language Models (LLMs) are advanced artificial intelligence systems that can understand and generate human language. They are trained on large amounts of text data and are used for tasks such as answering questions, summarizing text, and generating written content (Naveed et al., 2024).

Figure 3 shows a general overview of Large Language Models (LLMs). It explains the main stages of how LLMs are built, trained, tested, and used. The process starts with pre-training, where models learn from large amounts of text. Then, fine-tuning is used to improve the model for specific tasks using methods such as instruction learning and alignment. Figure 3 also shows efficient methods like quantization and pruning, which help reduce costs and improve speed. LLMs are evaluated using language understanding (NLU) and language generation (NLG) tasks. The diagram also highlights challenges, such as data bias, high cost, and safety risks. Finally, it shows how LLMs are used in many applications, including education, medicine, law, finance, and coding content (Naveed et al., 2024).

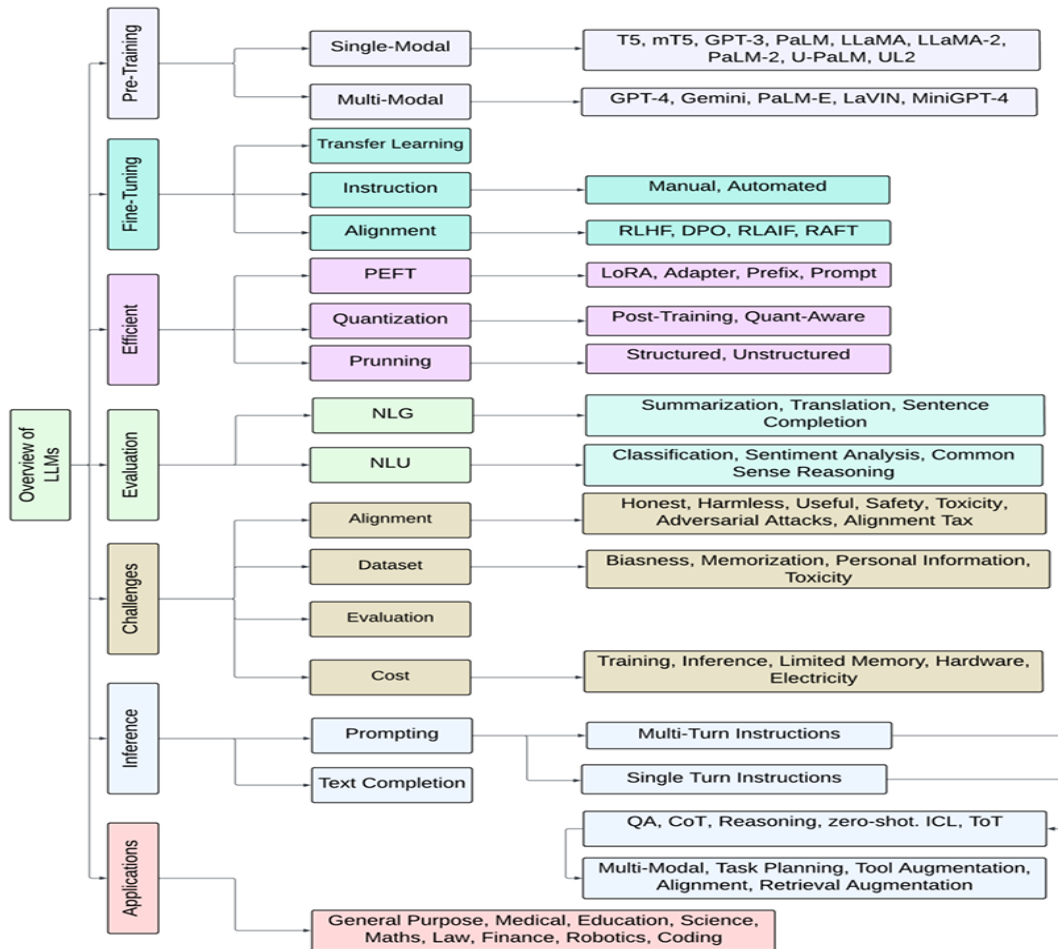


Figure 3 LLM Overview

(Naveed et al., 2024)

3.3 Advantages and Disadvantages of Artificial Intelligence

Advantages of Artificial Intelligence: According to Khanzode and Sarode (2020), Artificial Intelligence has several important advantages for improving tasks and services. AI can finish work faster than humans and can complete stressful or complex tasks more easily. It helps reduce errors, increases efficiency, and can handle many functions at the same time. Also, AI can calculate difficult situations, discover new information such as space exploration, and work with high accuracy in a short period (Khanzode & Sarode, 2020).

Disadvantages of Artificial Intelligence: The article also highlights some disadvantages. AI can sometimes be misused and may cause harm if used incorrectly. It may not always follow human commands accurately, and it can increase unemployment because machines replace human jobs. AI also lacks human creativity and emotional understanding. Young people may become dependent on technology, and developing AI systems requires a lot of time, money, and resources (Khanzode & Sarode, 2020). Also, some AI models do not provide transparency, which means they do not provide visibility and explainability of their decisions and therefore act like a “black box” (Gillani et al., 2023).

After explaining the main concepts, fields, and advantages and disadvantages of artificial intelligence, the next step is to show how these technologies are used in real auditing work. Artificial intelligence is now widely applied to support auditors in different audit tasks. The following chapter explains how AI is used in auditing activities.

4 AI-Driven Auditing

This chapter explains how artificial intelligence is used in auditing work. Different AI methods are applied to help auditors analyze data and detect risks. This chapter introduces the use of different AI techniques in auditing.

4.1 Generative AI for Auditing

This section focuses on one of the main Generative AI applications in auditing, which is the use of Large Language Models.

4.1.1 Large Language Models in Auditing Activities

LLMs can generate human-like responses, prepare audit working papers and reports, summarize auditing standards, and support audit planning. They help auditors by identifying risk areas, assisting in internal control assessments, and improving the efficiency and effectiveness of audit procedures. LLMs also save time by automating routine and repetitive tasks, which allows auditors to focus more on complex and professional judgment-based work (Fotoh & Mugwira, 2025).

4.2 Supervised Techniques used in Auditing Activities

Supervised learning is a machine learning method within artificial intelligence where models learn from labeled data to make predictions and classifications. In auditing, AI and ML techniques are used to analyze financial data, detect errors, classify transactions, and support accurate and efficient audit decisions.

4.2.1 Support Vector Machines

Support Vector Machine (SVM) is a classification and regression method based on statistical learning theory. It is used to analyze data and make predictions. An optimal hyperplane is constructed to separate data into two classes with the maximum margin. Non-linear transformations are applied to map data into a high-dimensional space, and the closest data points to the separating line are called support vectors. After training, new unknown data are classified into output classes. In auditing, SVM is used in financial applications such as financial statement

analysis, fraud detection, and risk prediction. It helps auditors classify companies and detect financial risk, and its performance is comparable or better than traditional methods like logistic regression (Nawaiseh et al., 2020).

The performance of the SVM model in auditing is evaluated using a classification test matrix (Figure 4), which compares actual audit opinions against those predicted by the model. The matrix captures four possible outcomes: True Positive (TP), where both the actual and predicted classifications are qualified; False Negative (FN), where a qualified company is incorrectly predicted as unqualified; False Positive (FP), where an unqualified company is incorrectly predicted as qualified; and True Negative (TN), where both the actual and predicted classifications are unqualified. These four outcomes collectively serve as the basis for assessing the accuracy and reliability of the SVM model in audit opinion classification (Nawaiseh et al., 2020).

$$\text{Average Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}$$

Equation 1 Accuracy using Confusion Matrix Variables

(Nawaiseh et al., 2020)

Actual Class (%)	Predicted Classifier (%)	
	<i>Qualified</i>	<i>Unqualified</i>
<i>Qualified</i>	True positive (TP)	False negative (FN)
<i>Unqualified</i>	False Positive (FP)	True Negative (TN)

Figure 4 Confusion Matrix

(Nawaiseh et al., 2020)

4.2.2 Decision Trees

Decision Tree is a supervised machine learning technique that classifies data by splitting it into groups using simple rules. It works by choosing the best variable at each step using mathematical measures. One important measure is **entropy**, which shows how mixed the data is, and it is calculated as

$$H(D) = -\sum p_i \log_2(p_i)$$

Equation 2 Entropy

(Apak & Ganji, 2025)

Another measure is **information gain**, which selects the best split:

$$IG(D, A) = H(D) - \sum \frac{|D_v|}{|D|} H(D_v)$$

Equation 3 Information Gain

(Apak & Ganji, 2025)

Also, the **Gini index** is used in some trees to measure impurity:

$$Gini(D) = 1 - \sum p_i^2$$

Equation 4 Gini Index

(Apak & Ganji, 2025)

In auditing, Decision Trees are used to predict audit quality and financial risk using financial variables such as debt-to-equity, return on assets (ROA), profit margin, stock volatility, and trading volume. The study shows that the C5.0 decision tree model achieved the highest accuracy (about 97–98%), which means it can correctly classify most companies into high or low audit quality. The most important factors were debt-to-equity and ROA, which shows that company risk and profitability strongly affect audit results (Apak & Ganji, 2025).

4.3 Unsupervised Techniques used in Auditing Activities

Unsupervised learning is a machine learning method within artificial intelligence where models analyze unlabeled data to discover patterns. In auditing, AI and ML unsupervised techniques are used to identify hidden trends, group similar transactions, and detect unusual or suspicious financial activities.

4.3.1 K-Means Clustering

K-means is an unsupervised machine learning method that groups data into similar clusters. First, k cluster centers (centroids) are selected. Each data point x_i is assigned to the nearest centroid using a distance measure, usually the Euclidean distance:

$$d(x_i, c_j) = \sqrt{(x_i - c_j)^2}$$

Equation 5 Euclidean distance

(Mehta et al., 2025)

Then, each centroid is updated as the **average (mean)** of the points in its cluster:

$$c_j = \frac{1}{n_j} \sum x_i$$

Equation 6 Mean.

(Mehta et al., 2025)

These two steps repeat until the clusters do not change. The goal of K-means is to minimize the total distance inside each cluster, called **the within-cluster sum of squares (WCSS)**:

$$WCSS = \sum_{j=1}^k \sum_{x_i \in S_j} \|x_i - c_j\|^2$$

Equation 7 within-cluster sum of squares (WCSS)

(Mehta et al., 2025)

In auditing, K-means is used to group similar companies or transactions based on financial features such as sales, expenses, receivables, liabilities, and transaction amounts. After clustering, the clusters that look quite different from normal groups are treated as suspicious and are selected for further audit investigation. This helps auditors in anomaly detection, fraud risk identification, and audit planning, especially when fraud labels are not available (Mehta et al., 2025).

4.3.2 Unsupervised NLP Anomaly Detection

Natural Language Processing (NLP) works by allowing computers to read, clean, and analyze human language in a systematic way. According to the article, the process begins with collecting financial text data from sources such as emails, financial reports, contracts, and audit documents. The text is then preprocessed through cleaning and removing unnecessary words to prepare it for analysis. After that, important linguistic features are extracted using sentiment analysis to detect misleading tone, named entity recognition (NER) to identify people, companies, and financial values, and topic modeling to discover suspicious themes in documents. These features are analyzed using advanced deep learning models such as BERT, which can recognize abnormal patterns in language. In anomaly detection, this method is applied to automatically identify unusual, misleading, or inconsistent financial communication, and the detected texts are flagged as potential fraud or risk cases. (Lan et al., 2025)

4.4 Deep Learning used in Auditing Activity

Deep learning is an advanced machine learning technique within artificial intelligence that analyzes large and complex datasets. In auditing, deep learning models help detect unusual patterns, hidden risks, and complex anomalies in financial data with high accuracy.

4.4.1 Autoencoder Neural Networks

An autoencoder is an unsupervised neural network that teaches the normal pattern of data by trying to reconstruct the original input. It has two main parts: an encoder, which changes the input data x into a compressed form, and a decoder, which rebuilds the data back to its original form so that $d(e(x)) \approx x$ as shown in Figure 5. The model is trained by minimizing the reconstruction error, measured using the Mean Squared Error (MSE), which is.

$$L(x,y) = \|x - y\|^2$$

Equation 8 Mean Squared Error (MSE)

(Schultz & Tropmann-Frick, 2020).

When the error is high, the data point is considered abnormal. In auditing, this method is applied to journal entry testing. The study used real ERP accounting data from three financial accounts (revenue domestic, revenue foreign, and expenses). The journal entry attributes were prepared using one-hot encoding for categorical data and standardization for numerical amounts. After training, journal entries with high errors were flagged as unusual. The detected results were then compared with manual decisions made by experienced external auditors, and the autoencoder achieved high recall and good F-scores, while also finding unusual entries that auditors had missed. This shows that autoencoders can strongly support auditors in detecting errors and potential fraud in large accounting datasets (Schultz & Tropmann-Frick, 2020).

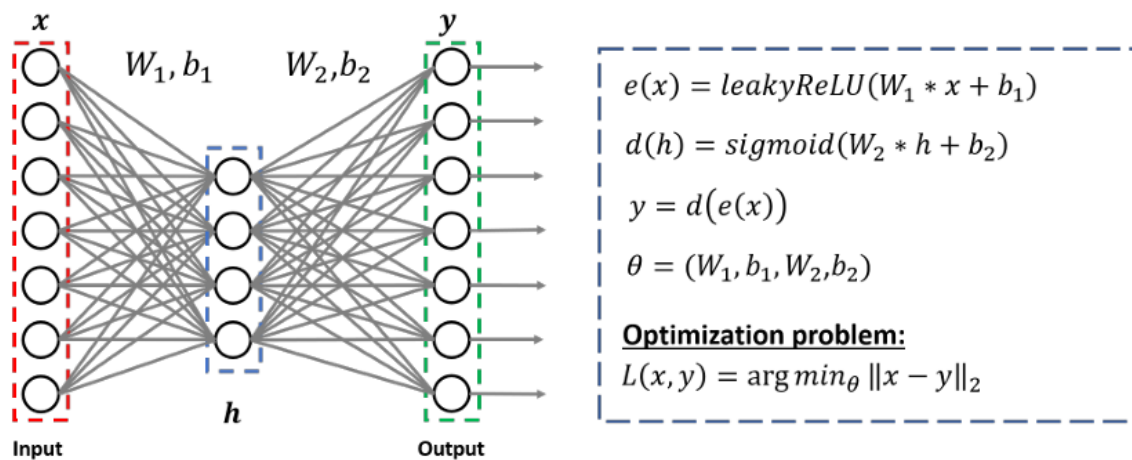


Figure 5 Architecture and Mathematical Representation of the Autoencoder Neural Network

(Schultz & Tropmann-Frick, 2020).

After explaining the foundations of auditing and the main concepts of artificial intelligence, and after describing how AI is used in auditing activities, it is important to review what recent studies have found in this field. The following chapter presents a comprehensive literature review that critically summarizes and compares previous studies on the application of artificial intelligence in financial auditing and identifies a relevant research gap.

5 Literature Review

This chapter is organized into two main parts. The first part, called "**State of the Art**", summarizes the most recent academic papers on **"Use of Artificial Intelligence in Financial Auditing to enhance Audit Quality"**. The second part, the **"Discussion"** section presents a comparative analysis of the selected studies by examining and comparing their main ideas, research methods, and key findings. This comparison helps to highlight similarities and differences between the studies and allows for a deeper understanding of current research trends. Through this systematic comparison, the remaining weaknesses and missing areas in the literature are identified, which helps to clearly define the research gap.

5.1 State of the Art

This section begins by outlining the process used to identify the most relevant academic papers on the topic **"Use of Artificial Intelligence in Financial Auditing to enhance Audit Quality."** The search strategy was based on a set of targeted keywords **"Artificial Intelligence," "AI," "Audit Quality," "Fraud Detection," "AI in Auditing,"** and **"Financial Auditing"**. These keywords were chosen to ensure that the search captured studies that directly examined how AI supports the detection of misstatements, anomalies, and irregularities within financial audit processes. **Google Scholar** and **Research Gate** served as the primary search platform, with the search restricted to papers published between **2021 and 2025** to ensure the inclusion of recent and up-to-date academic contributions.

Applying these keywords resulted in **Eleven** academic papers. After removing implementation of **post filtering by duplicate entries and excluding papers that lacked sufficient depth or clear relevance** to the topic, **five** papers remained. These final papers were all **peer-reviewed publications from reputable journals or conferences**.

The selected papers are reviewed individually to highlight their goals and key contributions. The following summaries outline the most relevant findings that shape the current understanding of AI in financial auditing.

Technological Convergence in Financial Auditing (Dos Santos & Dos Santos, 2025)

This specific combination of keywords (**auditing, artificial intelligence, deep learning, machine learning, audit quality**) was used to find the first paper because they directly match its main topic and scope. The words auditing and audit quality were used to target studies related to the audit field and its improvement. The terms artificial intelligence, machine learning, and deep learning were added to capture the main smart technologies discussed in modern audit research.

Goal:

The aim of the study was to conduct an SLR (**Systematic Literature Review: a structured and organized review of many academic papers** using clear rules to reduce bias) to examine how artificial intelligence technologies contribute to improving the financial auditing process by focusing on how these key technologies (AI,ML,DL) enhance audit quality, improve fraud detection, increase efficiency, and strengthen the reliability of financial reporting. The paper also aimed to answer its research question: What are the contributions of AI, deep learning, and machine learning to financial auditing, and what possibilities exist for future research? Therefore, its goal is to provide researchers and auditors with a foundation for understanding technological convergence in modern audit practices.

Gap:

The gap addressed in this research is that no studies examine the effect of key AI technologies on the auditing quality process as a whole; existing research focuses only on one technology at a time. The study also highlights how these technologies affect audit quality, fraud detection, audit efficiency, and the challenges faced by SMEs when adopting AI-driven auditing. Additionally, the paper points out that there is still a shortage of empirical evidence comparing technology-supported audits with traditional human-performed audits regarding trust and efficiency.

Methods Used:

The study followed the PRISMA approach, which provides transparent and reproducible procedures for systematic reviews. This method helps minimize bias and ensures that results are clear, structured, and reliable. The review was conducted in four main steps, as shown in Figure 6.

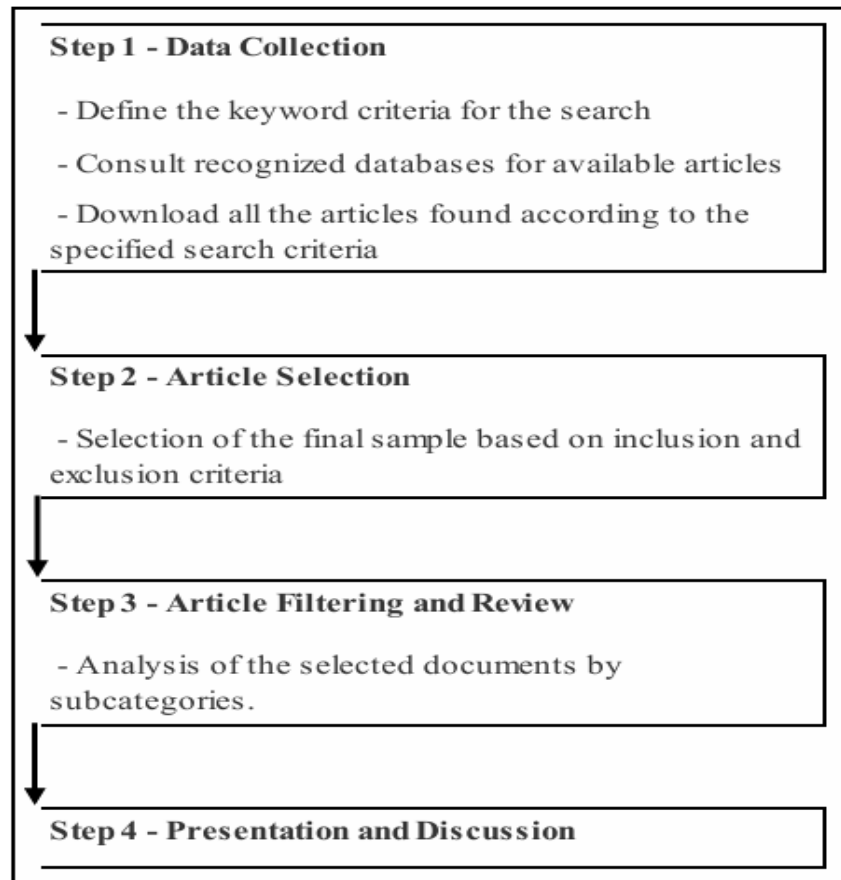


Figure 6 PRISMA Approach
(Dos Santos & Dos Santos, 2025)

Limitations:

The study is restricted to one database (Web of Science), and despite applying the PRISMA approach with clearly defined inclusion and exclusion criteria, the selection and categorization of articles still involve human judgment, meaning some level of bias may have influenced the final sample.

AI Opportunities and Challenges in Financial Auditing (Hamzah et al., 2024)

Goal:

This paper's goal is to identify how AI can be integrated into the auditing process and to outline the opportunities AI provides, such as improving audit efficiency, accuracy, anomaly detection, fraud detection, and decision-making.

Gap:

The paper addresses the lack of a focused and integrated analysis that explains both the opportunities and challenges of adopting AI in financial auditing, specifically the missing discussion on auditor skill gaps, AI transparency issues, ethical and bias risks, data security concerns, and the absence of clear regulatory standards.

Methods Used:

This study uses a qualitative literature review to explore how AI is integrated into financial auditing. It relies on secondary data from journals, industry reports, regulatory documents, and sources accessed through Google Scholar, ScienceDirect, and IAASB publications. Relevant literature was selected based on credibility, relevance, and recency, then categorized into themes such as opportunities, challenges, and solutions. Using content analysis, the study identified how AI can enhance the audit process, the obstacles to its adoption, and potential ways to address these challenges in the era of the Industrial Revolution 4.0.

Limitations:

This study has several limitations. It uses only information from books, journals, and reports, so AI is not tested in real audit work. As a result, the findings depend entirely on secondary sources. The paper also discusses issues such as low auditor skills, AI transparency, and data security based only on past studies rather than real cases, meaning the findings may not fully represent how AI performs in actual auditing environments.

Artificial Intelligence and Audit Quality (Wijaya et al., 2025)

Goal:

This study aims to examine how artificial intelligence influences audit quality through a comprehensive empirical literature review using the Scopus database. The paper aims to summarize existing evidence on the overall impact of AI adoption in auditing, focusing on how AI improves audit effectiveness, efficiency, accuracy, fraud detection, and decision-making. It also highlights areas where further research is required.

Gap:

The authors point out that there is no comprehensive empirical literature review that gathers, compares, and evaluates the findings from empirical studies on AI's impact on audit effectiveness, accuracy, fraud detection, efficiency, and decision-making. Existing studies are fragmented or examine limited variables.

Methods Used:

This study uses a qualitative literature review approach to assess how AI affects audit quality. All data were collected from Scopus papers to ensure academic rigor. The authors searched using the keyword “Artificial Intelligence on Audit Quality,” with a specific focus on the 2010–2025 time limit and only included English-language, peer-reviewed, fully published scientific journal articles. This process resulted in 325 articles, which were analyzed in the study.

Limitations:

This study has several limitations. Many auditors do not have sufficient skills to use AI tools, which limits effective application in real audit work. Ethical and legal concerns also remain, such as privacy, fairness, and assigning responsibility for AI-supported decisions. Most research originates from developed countries, meaning the findings may not represent less-developed regions. Additionally, many studies rely on past data or simulations rather than long-term real audit engagements, resulting in uncertainty regarding AI performance in practical audit environments. Existing literature also does not always explain how AI reaches specific outputs, creating uncertainty when applying such systems in auditing contexts.

Artificial Intelligence in Auditing: Opportunities, Challenges, and Future Directions (Firat, 2025)

“Risk Assessment” keyword was used to find this article to target studies which shows how AI is used to improve audit risk evaluation.

Goal:

This paper examines how AI is transforming auditing from several different angles. It highlights the opportunities, challenges, and future development of AI. As shown in Figure 7 this paper analyzes how AI improves risk assessment, supports better decision-making, and automates audit processes, while also addressing the difficulties that arise with its use, the study offers a comprehensive understanding of AI’s growing influence on the future of auditing.

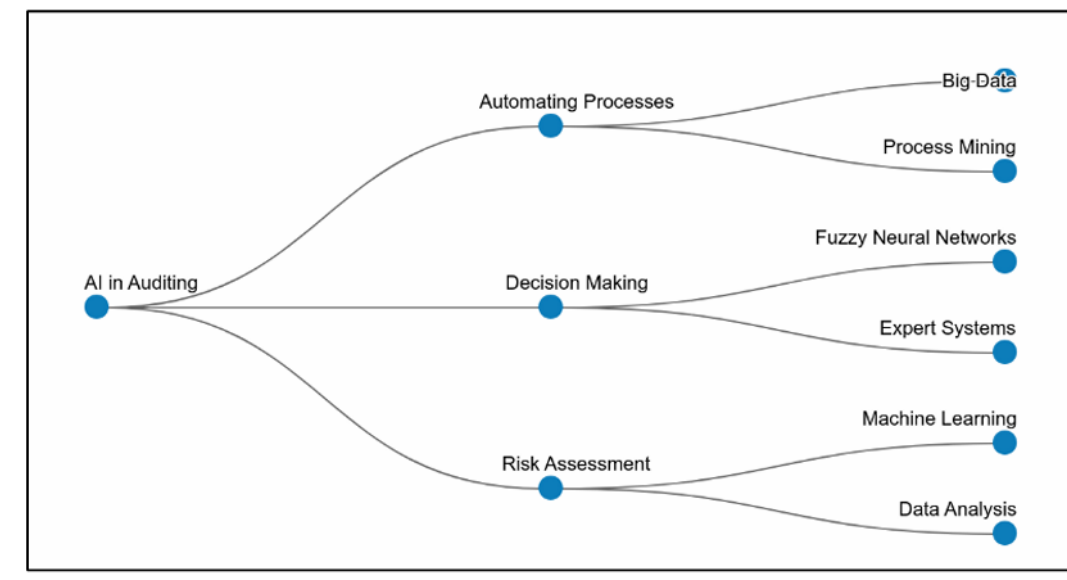


Figure 7 Concept Map of AI Opportunities in Auditing

(Firat, 2025)

Gap:

The gap addressed in this paper is to explain how AI affects risk assessment, anomaly detection, and continuous auditing at the same time. There is also not enough work that guides auditors and regulators on the required skills, rules, and ethical practices when using AI.

Methods Used:

The study uses a qualitative method, meaning information is collected from written sources. The author reads and analyzes secondary data such as academic papers, industry reports, and case studies. The paper reviews past research about AI in auditing and groups the information under themes such as risk assessment, anomaly detection, continuous auditing, and ethics.

Limitations:

The study has several limitations. It uses only secondary data, so AI is not tested in real auditing situations. Because the results depend entirely on what other researchers wrote, the findings may not fully represent real practice. In addition, the study does not include interviews or surveys with auditors, meaning practical insights and real-world experiences may be missing.

AI Opportunities and Challenges in Financial Auditing (Onyenahazi, 2025)**Goal:**

The goal of the study is to understand how AI can make financial audits more accurate, faster, and more effective. The paper also aims to examine how AI tools support data checking, fraud detection, and improved risk assessment and to explain the problems and risks associated with using AI in audits, such as fairness and transparency concerns. Finally, the study provides a plan for how companies and auditors can use AI safely and responsibly in audit work.

Gap:

The paper explains that current audits still rely on traditional methods that cannot handle large, complex, and fast-changing financial data. These traditional methods depend on manual work, small samples, and fixed rules, causing audits to miss new or hidden problems. Existing computer tools also lack the ability to clearly explain their decisions. In addition, there is no global standard for using AI in audits, and issues such as bias, unclear decision processes, and privacy risks remain unresolved.

Methods Used:

The study uses a mixed-methods approach to provide a comprehensive understanding of AI's role in auditing. First, a review of academic literature, industry publications, and international audit standards establishes the theoretical foundation. Second, qualitative interviews with auditors, data scientists, and compliance experts provide insights into practical challenges and real-world applications. Third, the paper presents case studies showing how large firms, public sectors, and

multinational corporations apply AI in their audits, including examples of fraud detection and continuous monitoring. Finally, a comparative analysis examines differences in AI adoption across industries and regulatory environments, helping identify patterns, limitations, and areas for improvement.

Limitations:

The study mentions several limitations related to the use of AI in financial auditing. Many AI models lack transparency, making it difficult for auditors to understand how conclusions are generated. AI systems may also be biased, leading to unfair or inaccurate results. There is also uncertainty about who is responsible when AI systems make errors. Smaller firms may face additional limitations due to limited resources or lack of technical expertise. Moreover, strong data privacy measures are required when handling sensitive financial information.

5.2 Discussion

This section provides a comparative analysis of the state of the art in AI-driven auditing, synthesizing research findings from 2021–2025. The purpose of this discussion is to highlight similarities and differences in the goals, gaps, methods used, and limitations, resulting in the identification of the remaining scientific gap in the field.

Goal

Dos Santos & Dos Santos (2025) discuss a wide goal. It studies how these technologies (AI, machine learning, and deep learning) all support the auditing process. Compared to other papers, this goal is the broadest, because the study examines many tools at the same time instead of focusing on only one topic.

Hamzah et al. (2024) have a more practical goal. The goal is to explain how AI can be added to the audit process and what new opportunities and problems appear when AI is used. Compared to Dos Santos & Dos Santos (2025), this goal is narrower and more focused, because the paper focuses only on AI and not on other technologies. This goal is also more practical than theoretical.

Wijaya et al. (2025) have a more focused goal because the aim is to examine real evidence from Scopus papers to see whether AI improves audit quality. Compared to Hamzah et al. (2024), this goal is more research-based rather than paper-based. Compared to Dos Santos & Dos Santos (2025), this goal is narrower because the focus is only on audit quality and AI, not on multiple technologies or the entire auditing process.

Firat (2025) has a future-oriented goal. The goal is to explain how AI is changing auditing and how it may shape the future, especially in risk assessment, decision-making, and automation. Compared to Wijaya et al. (2025), this goal is less about evidence and more about future direction. Compared to Hamzah et al. (2024), this goal is wider because the paper looks not only at current challenges but also at future developments.

Onyenahazi (2025) has a system-improvement goal. The aim is to show how AI can make audits more accurate, faster, and more compliant by examining how AI tools (machine learning, NLP) are used in key audit tasks such as journal entry testing, contract review, inventory checks, and financial reporting. Compared to Firat (2025), this goal is more applied and technical. Compared to Wijaya et al. (2025), this goal does not focus on reviewing evidence but on explaining how AI improves specific audit procedures. Compared to Hamzah et al. (2024), this goal goes deeper into operational examples, not only opportunities and challenges.

Taken together, the goals move from extremely broad Dos Santos & Dos Santos (2025), to practical Hamzah et al. (2024), to evidence-based Wijaya et al. (2025), to future-focused Firat (2025), and finally to operational and system-improvement Onyenahazi (2025).

Among the five studies, the goals of Wijaya et al. (2025), Hamzah et al. (2024), and Onyenahazi (2025) are clearly met because the papers provide focused evidence, practical analysis, and clear examples that directly support their research aims. In contrast, Firat (2025) only partly meets the goal because the future-oriented discussion is broad and not strongly supported with evidence. Dos Santos & Dos Santos (2025) also do not fully meet the wide goal, as studying combination technologies at once makes the analysis general rather than detailed. Overall, the more focused studies achieve their goals, while the broader ones only partially do.

Gap

Dos Santos & Dos Santos (2025) try to fill an extremely broad gap. Past studies examined only one technology at a time, but no study examined this combination of technologies together (AI, machine learning, deep learning) and how these tools affect audit quality, fraud detection, and efficiency. There is also a gap in research into how SMEs struggle when trying to use these tools. Another gap is the lack of studies comparing technology-based audits with human audits for trust and efficiency. This gap is the widest because it connects multiple technologies with multiple auditing challenges.

Hamzah et al. (2024) try to fill a different gap. The missing part in the literature is a combined explanation of both the opportunities and challenges of adopting AI in financial auditing. Past research discussed benefits or problems, but not together in one structured way. There are also gaps related to auditor skill shortages, AI transparency issues, ethical risks, data privacy problems, and missing regulations. Compared to Dos Santos & Dos Santos (2025), this gap is narrower because the focus is only on AI, but deeper because it examines real adoption challenges.

Wijaya et al. (2025) try to fill a gap in empirical evidence. The authors state that there is no complete empirical review that collects, compares, and evaluates the results from past studies on AI's real impact on audit quality. Compared to Hamzah et al. (2024), this gap is more research-based because it focuses on missing empirical data rather than missing practical guidance.

Firat (2025) tries to fill a future-oriented gap. There is no full review explaining how AI affects risk assessment, anomaly detection, and continuous auditing at the same time. There is also a lack of guidance for auditors and regulators on skills, ethical rules, and governance when using AI. Compared to Wijaya et al. (2025), this gap is less about missing evidence and more about missing guidance and future needs.

Onyenahazi (2025) tries to fill an operational and technical gap. The missing part in the literature is a clear explanation of how AI tools function inside real audit procedures, such as how machine learning and NLP are used in journal entry testing, contract review, inventory verification, and anomaly detection. Previous studies mentioned these technologies but did not provide a detailed, step-by-step explanation of how these tools improve accuracy, efficiency, and compliance in practice.

The gaps move from very broad and multi-technology Dos Santos & Dos Santos (2025), to practical AI adoption problems Hamzah et al. (2024), to missing empirical evidence Wijaya et al. (2025), to missing guidance for future skills and ethical needs Firat (2025), and finally to missing procedure-level explanations of how AI operates inside real audit tasks Onyenahazi (2025).

Methods Used

Dos Santos & Dos Santos (2025) use a PRISMA systematic review method. This method has clear steps that must be followed and helps reduce bias by making the review transparent. Four main steps are used to collect, filter, and analyze studies. Compared to the other papers, this is the most structured and formal method because PRISMA is a strict guideline for systematic reviews.

Hamzah et al. (2024) use a qualitative literature review. Information is collected from journals, reports, and regulatory documents found on Google Scholar, ScienceDirect, and IAASB sources. Papers are selected based on credibility and relevance, and the findings are grouped into themes such as opportunities, challenges, and solutions. Compared to Dos Santos (2025), this method is simpler and less strict because PRISMA is not used and general content analysis is applied instead.

Wijaya et al. (2025) use a qualitative literature review, but the method is more specific. Only Scopus-indexed empirical papers from 2010–2025 are collected. One keyword is used, and the data is limited to English, peer-reviewed, fully published articles. The process starts with 325 papers. Compared to Hamzah et al. (2024), this method is more selective and academic, relying on Scopus as a high-quality database. Compared to the others, it is still qualitative but more focused on empirical studies.

Firat (2025) uses a qualitative review of secondary data. Academic papers, industry reports, and case studies are analyzed and organized into themes such as risk assessment, anomaly detection, continuous auditing, and ethics. Compared to Wijaya et al. (2025), this method is more general and not limited to a specific database, meaning it provides a wide view but is less strict in selection.

Onyenahazi (2025) uses a mixed-methods approach. Literature reviews, interviews with auditors and data scientists, and real case studies are combined. The paper also examines how AI tools (ML, NLP) work in real audit steps. Compared to the others, this method is the most practical because it combines theory with real-world insight and demonstrates how AI functions inside actual audit procedures.

Overall, Dos Santos & Dos Santos (2025) have the strongest and most rigorous method because PRISMA is a formal, standardized, and transparent framework that reduces bias. Wijaya et al. (2025) come next because the Scopus-based empirical selection is selective and academically strong. Onyenahazi (2025) is the best for practical insights because real cases and interviews are used. Hamzah et al. (2024) and Firat (2025) use simpler qualitative methods, which are useful for broad discussions but are less strict and less evidence-based compared to PRISMA and Scopus methods.

Limitations

Dos Santos & Dos Santos (2025) have clear limitations. The study uses only one database (Web of Science), which means important studies from other databases may be missing. Even though the PRISMA steps are followed, article selection still depends on human judgment, so some bias may occur. Compared to the other papers, this limitation is related to the research process and sources, not auditing or AI issues.

Hamzah et al. (2024) also have important limitations. The study uses only books, journals, and reports, so AI is not tested in real audit work. All findings depend on secondary sources, which may not represent actual company practices. Compared to Dos Santos (2025), this limitation is more practical because nothing is tested directly.

Wijaya et al. (2025) face limitations related to real-world use. Many auditors lack sufficient AI skills, so AI is not used fully in practice, affecting the studies reviewed. There are also ethical and legal issues such as privacy and fairness that remain unresolved. Most empirical studies come from developed countries, meaning the findings may not represent other regions. Compared to Hamzah et al. (2024), these limitations are more related to global variation and skill gaps.

Firat (2025) has limitations similar to Hamzah et al. (2024). The study uses only secondary data, so AI is not tested in real audits. The results depend on past papers, which may not reflect real practice. Compared to Wijaya et al. (2025), this limitation is simpler because the focus is only on missing real-life input, not on global or ethical issues.

Onyenahazi (2025) also has important limitations. The method depends heavily on interviews and selected case studies, which may not represent all audit environments or industries. Most examples come from large firms, so the findings may not apply to smaller companies. Another limitation is that the paper explains how AI works in audit steps but does not measure the performance of these tools using real data. Compared to Fırat (2025), this limitation is more about representativeness and missing empirical measurement. Table 1 highlights similarities and differences in goals, gaps, methods used, and limitations.

Paper (Author & Year)	Research Design Approach	AI & Auditing Focus Area	Explainable AI / Transparency Perspective
Dos Santos & Dos Santos (2025)	Strict PRISMA SLR with search strings	AI, machine learning, and deep learning in financial auditing	Discusses audit transparency, ethical and bias-mitigation concerns, and effects on auditor independence; no concrete XAI model proposed.
Hamzah et al. (2024)	Qualitative literature review	AI opportunities and challenges in auditing	Highlights algorithm transparency issues and the “black box” problem; calls for more explainable and understandable AI systems.
Wijaya et al. (2025)	Empirical review of Scopus studies	Impact of AI on audit quality especially (fraud detection, accuracy, efficiency)	Identifies interpretability concerns and stresses the importance of explainable AI (XAI) for ethical governance and auditor trust.
Fırat (2025)	Conceptual review	AI in risk assessment, anomaly detection, and continuous auditing	Emphasizes the need for explainability and security in AI systems to address ethical concerns.
Onyenahazi (2025)	Case studies, qualitative interviews, and literature review	AI in financial auditing to improve accuracy, fraud detection, and regulatory compliance	Discusses model explainability, governance standards, and risks of black- box decision-making in audit automation.

Table 1 Summary of Empirical Studies on AI in Financial Auditing

5.3 Scientific Gap

The analysis of the literature shows that AI can improve financial auditing, but a major problem remains: many AI systems are difficult to understand. These systems work like a “black box,” meaning auditors cannot easily see how the AI makes its decisions or why something is identified as fraud or an error. Because of this, AI results cannot be fully trusted or explained, which is required in real audits. The reviewed studies also do not offer clear methods or tools to help auditors check or justify AI outputs. This creates a research gap, as there is still a lack of practical artifacts that show how AI can be used in a transparent and reliable way inside the audit process.

Research Question:

How can Explainable AI (XAI) be integrated into financial auditing to ensure that AI outputs are transparent, understandable, and supportive of auditors’ decisions?

Research Objectives

The main objective of this study is to **develop a conceptual model** for integrating Explainable Artificial Intelligence (XAI) into the financial auditing process, addressing the identified scientific gap, through which audit transparency, interpretability, and reliability can be enhanced to improve the overall quality of AI-supported audit processes.

To achieve the research objectives presented, an appropriate research methodology is required. Therefore, the following chapter introduces several scientific research approaches. Based on this overview, the most suitable methodology for this study will be selected and justified.

6 Scientific Methodology

In this chapter, an overview of several research methodologies is presented, including Design Science, Data Science, and quantitative and qualitative research approaches. The appropriate method or combination of methods is then selected for the proposed solution approach, based on the artifact type defined in the research objectives.

6.1 Design Science

Hevner et al. (2004) establish Design Science Research (DSR) as a legitimate paradigm in Information Systems, distinguishing it from behavioral science by focusing on creating innovative artifacts to solve business problems. DSR produces IT artifacts, including constructs (conceptual vocabulary), models (relationships or structures), methods (procedures or algorithms), and instantiations (implemented systems or prototypes). As illustrated in Figure 8 the framework is based on the interaction between two domains: the environment (people, organizations, and technology) ensuring relevance, and the knowledge base (theories and prior research) ensuring rigor. The design cycle connects these two domains through iterative building and evaluation activities. As a result, DSR must achieve a balance between practical relevance and scientific rigor (Hevner et al., 2004).

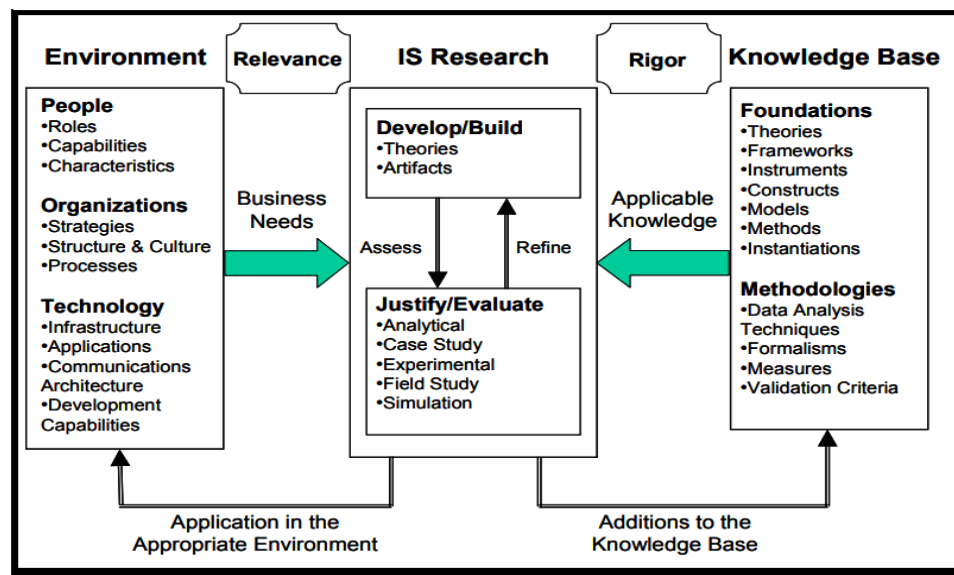


Figure 8 Information Systems Research Framework

(Hevner et al., 2004)

Hevner et al. formulates seven guidelines to define high-quality Design Science Research, as presented in Table 2. These guidelines specify what qualifies as rigorous and publishable DSR and establish it as a structured, theory-grounded, and evaluable paradigm in Information Systems (Hevner et al., 2004).

Guideline	Description
Guideline 1: Design as an Artifact	Design-science research must produce a viable artifact in the form of a construct, a model, a method, or an instantiation.
Guideline 2: Problem Relevance	The objective of design-science research is to develop technology-based solutions to important and relevant business problems.
Guideline 3: Design Evaluation	The utility, quality, and efficacy of a design artifact must be rigorously demonstrated via well-executed evaluation methods.
Guideline 4: Research Contributions	Effective design-science research must provide clear and verifiable contributions in the areas of the design artifact, design foundations, and/or design methodologies.
Guideline 5: Research Rigor	Design-science research relies upon the application of rigorous methods in both the construction and evaluation of the design artifact.
Guideline 6: Design as a Search Process	The search for an effective artifact requires utilizing available means to reach desired ends while satisfying laws in the problem environment.
Guideline 7: Communication of Research	Design-science research must be presented effectively both to technology-oriented as well as management-oriented audiences.

Table 2 Design-Science Research Guidelines

(Hevner et al., 2004)

While Peffers et al. (2007) proposed a structured Design Science Research Methodology (DSRM), addressing the lack of a clear process model for conducting and presenting DSR. The methodology provides a process model, aligns with prior literature, and offers a mental model for structuring and evaluating research outputs (Peffers et al., 2007).

DSRM consists of six sequential activities as shown in Figure 9: problem identification and motivation, defining solution objectives, design and development, demonstration, evaluation, and communication. The process begins with defining the research problem and deriving quantitative or qualitative objectives. An artifact is then designed and developed, demonstrated through methods such as experiments or case studies, and evaluated by comparing results with objectives using various assessment methods. Iteration may occur based on evaluation results. The final stage

ensures clear communication of the problem, artifact, rigor, novelty, and effectiveness. Although presented sequentially, the process allows entry at different points, including problem-centered, objective-centered, design-centered, and client-initiated approaches. The main contribution is providing a clear, practical, step-by-step methodology for conducting and reporting DSR (Peffer et al., 2007).

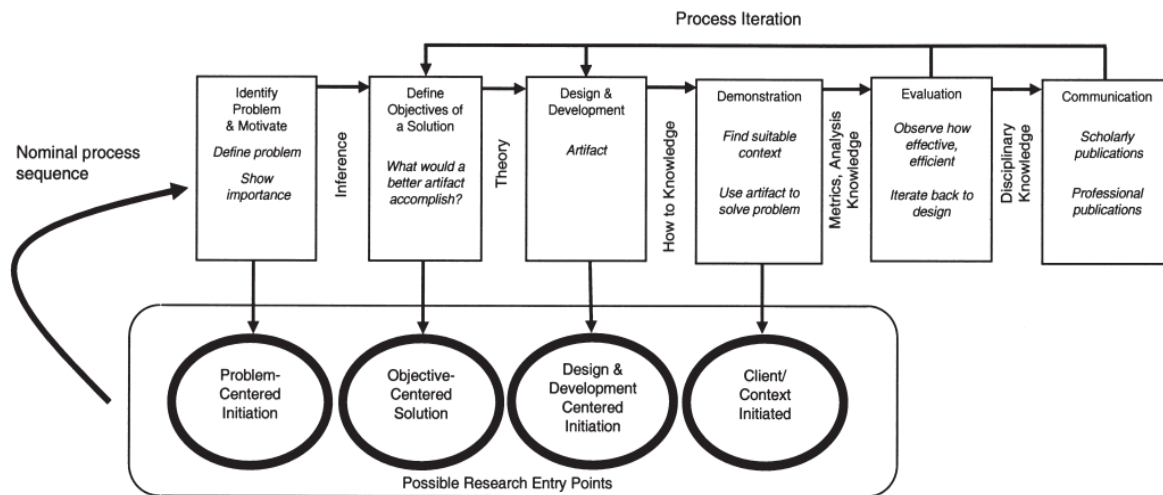


Figure 9 DSRM Process Model

(Peffer et al., 2007)

Offermann et al. (2009) propose a formalized Design Science Research (DSR) process model to strengthen methodological rigor while maintaining practical relevance, addressing the need for a structured and transparent research process. The model provides a systematic framework that integrates hypothesis formulation and combines qualitative and quantitative evaluation methods (Offermann et al., 2009).

Figure 10 shows that the process consists of three phases: Problem Identification, Solution Design, and Evaluation. The first phase defines and validates the research problem through literature review and expert input, and includes formulating an initial hypothesis linking the problem, artifact, and expected improvement. The second phase focuses on developing the artifact (method, model, framework, or system), grounded in theory, with documented design decisions and possible refinement of the hypothesis. The final phase evaluates the artifact by testing refined sub-hypotheses using methods such as case studies, experiments, or surveys, and comparing results with objectives to assess effectiveness and contribution. The main contribution is the formalization

of DSR into a structured, hypothesis-driven process that connects artifact development with rigorous evaluation (Offermann et al., 2009).

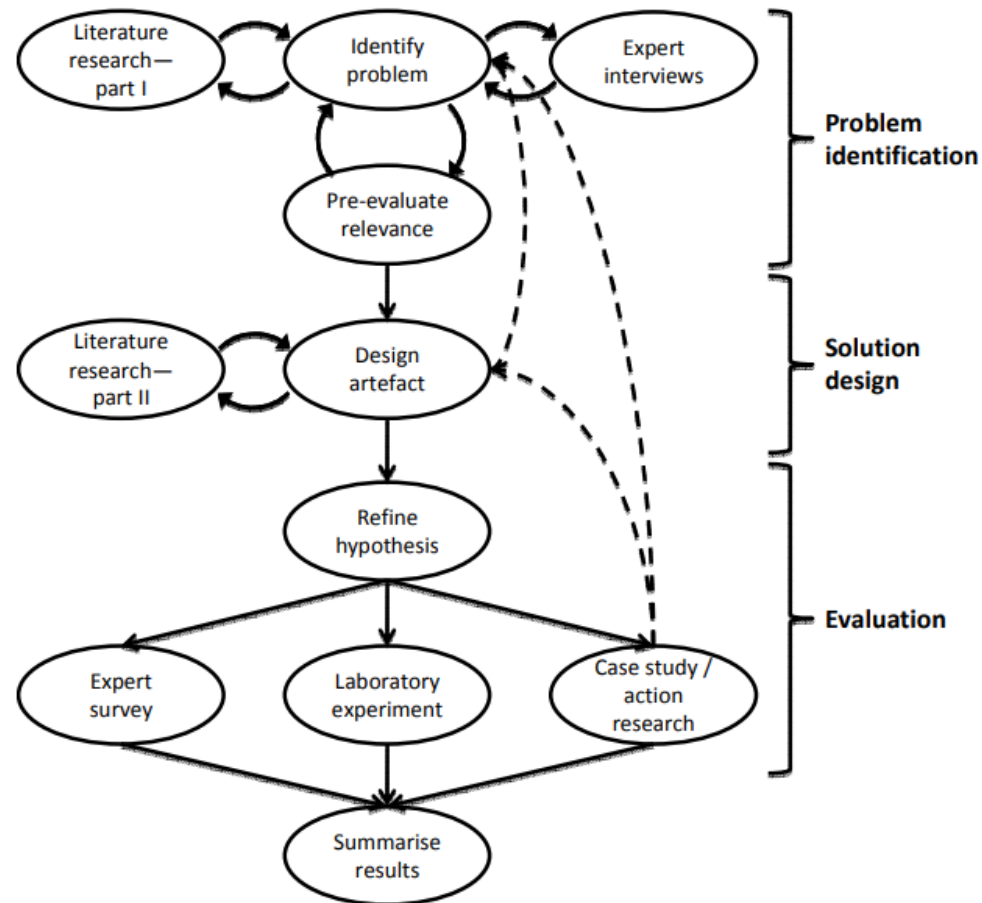


Figure 10 Proposed research process from Offermann

(Offermann et al., 2009)

The three Design Science Research approaches address different aspects of the methodology. Hevner et al. (2004) establish the theoretical foundation of DSR and define its principles and guidelines. Ken Peffers et al. (2007) translate these principles into a structured six-step research process. Offermann et al. (2009) further strengthen the methodology by introducing a hypothesis-driven and rigorous evaluation process.

Together, these models provide complementary guidance for conducting structured and scientifically rigorous Design Science Research

6.2 Data Science

CRISP-DM is a widely adopted, standardized methodology for structuring data science projects, organizing the end-to-end process into six iteratives. As shown in Figure 11, the framework begins with business understanding, where the problem, objectives, and success criteria are defined, followed by data understanding, which involves exploring collected data to assess its structure and quality. Data preparation, typically the most time-consuming phase, transforms raw data into a usable format through cleaning, integration, and formatting, after which the modeling phase applies and fine-tunes various techniques to identify the best solution. The evaluation phase then verifies that results align with the original business objectives, and deployment concludes the cycle by delivering outcomes through reports, visualizations, or automated systems. Due to its iterative nature, phases can be revisited as new insights emerge, making CRISP-DM a structured yet flexible framework that effectively supports communication, planning, and documentation across industries (Wirth & Hipp, 2000).

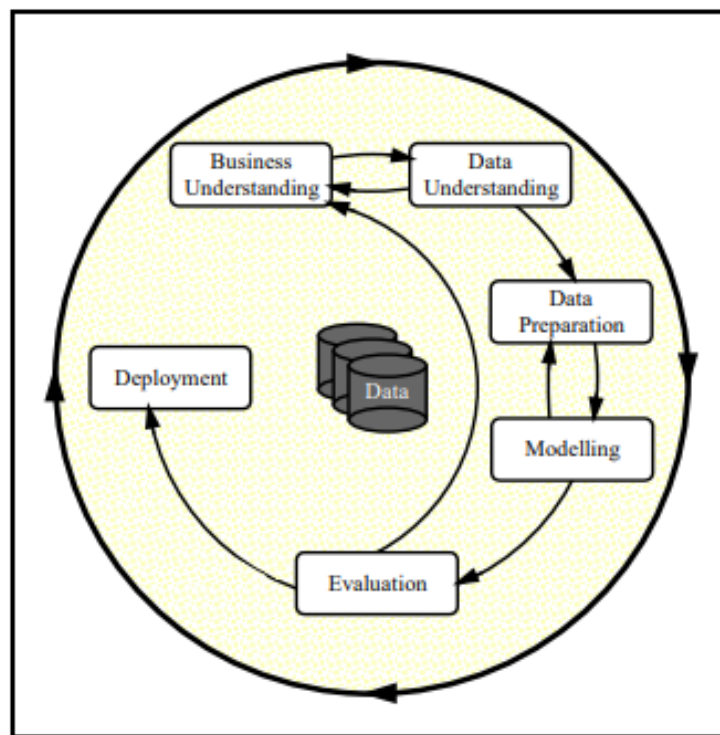


Figure 11 Phases of CRISP-DM Process Model for Data Mining
(Wirth & Hipp, 2000)

While CRISP-DM covers the full project lifecycle, Knowledge Discovery in Database (KDD) focuses on the analytical process of discovering knowledge from data.

KDD is defined as a systematic process for extracting valid, novel, useful, and understandable patterns from large datasets. Data mining is one step within KDD, supported by preparation, transformation, and interpretation stages to ensure reliable results. As shown in Figure 12, KDD consists of iterative stages: understanding the application domain, selecting relevant data, cleaning, and preprocessing, transforming, and reducing data, applying data mining algorithms, evaluating, and interpreting patterns, and presenting knowledge for decision-making (Fayyad et al., 1996).

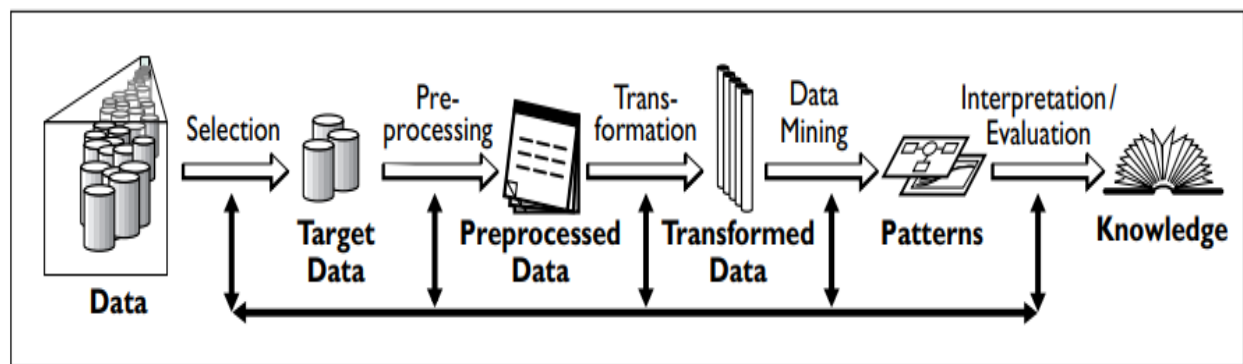


Figure 12 KDD Process

(Fayyad et al., 1996)

6.3 Quantitative and Qualitative

Scientific research in information systems applies both qualitative and quantitative methodologies depending on the problem and data type. These approaches differ in objectives, data collection, and analysis, but both contribute to reliable outcomes (Relacion, n.d.).

Qualitative research focuses on understanding phenomena through interpretation and contextual analysis using descriptive data from interviews, observations, documents, or open-ended questionnaires. It involves organizing data, identifying themes, and interpreting patterns, and is useful when knowledge is limited and deeper insights are needed. Quantitative research focuses on numerical measurement and statistical analysis using structured data collection methods such as surveys. It aims to test hypotheses and produce generalizable results. The main difference is that qualitative research emphasizes interpretation and detailed understanding, while quantitative research emphasizes measurement and statistical testing (Relacion, n.d.).

The Delphi method combines both approaches by collecting and refining expert opinions through iterative questionnaire rounds as shown in Figure 13. It begins with open-ended responses for qualitative insights, followed by structured evaluation for quantitative analysis. This process supports consensus building and systematic knowledge capture, making it suitable for areas with limited or evolving knowledge (Skulmoski et al., 2007).

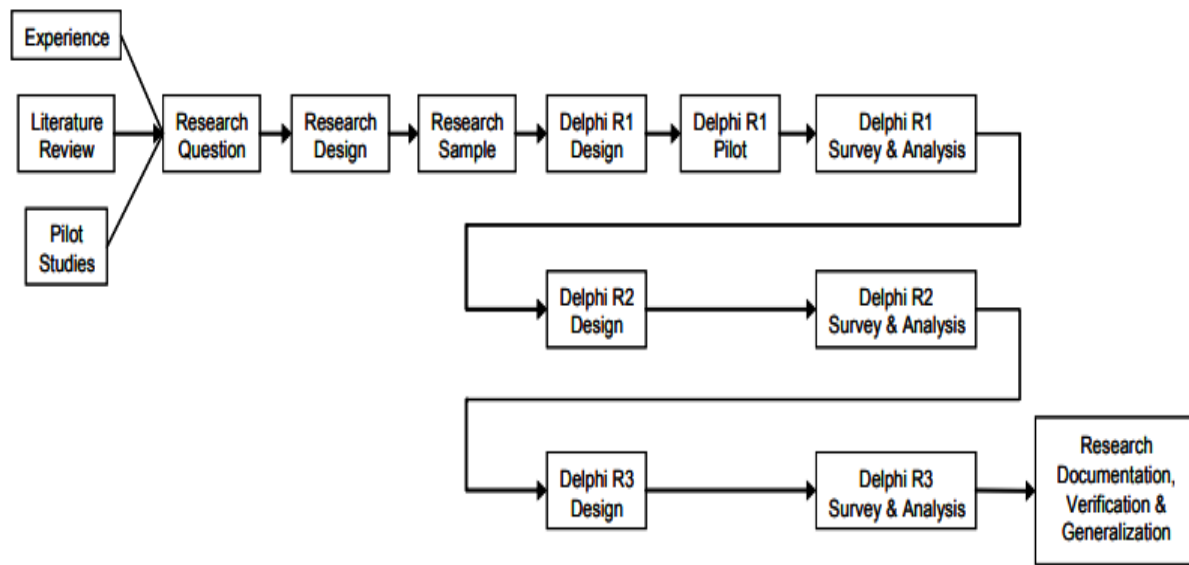


Figure 13 Three Round Delphi Process

(Skulmoski et al., 2007)

6.4 Literature Review

A Literature Review is a systematic, explicit, and reproducible process used to identify, evaluate, and interpret existing research within a specific field. Its main purpose is to support research development by identifying patterns, gaps, and key topics. Literature reviews may use qualitative, quantitative, or structured approaches, often applying structured content analysis. The process involves several stages: collecting relevant material using defined search strategies and criteria, filtering studies through titles and abstracts, and examining full papers (Fink, 1998).

A descriptive analysis is then conducted to identify key characteristics such as publication trends. Next, categories are created based on research questions to organize the literature into themes. Finally, studies are evaluated within these categories to identify common findings, challenges, and research gaps, which are interpreted through discussion and comparison (Fink, 1998).

6.5 Solution Approach

In this section, the solution approach adopted in this research is presented. Since the primary objective of this paper is the **development of a conceptual model**, Design Science Research (DSR) is identified as the most suitable methodology, as it is specifically oriented toward the construction and evaluation of novel artifacts that address real-world problems. The DSR framework proposed by Offermann et al. (2009) is adopted, as it provides a structured and iterative process for designing, implementing, and evaluating a scientifically grounded artifact that addresses the identified research gap, as illustrated in Figure 14.

Problem Identification

The first phase constitutes the foundation of the research. A structured literature review is conducted, within which a state of the art is established. Relevant prior papers are examined within a defined time frame, using specific keywords selected based on the research topic. For each included paper, four elements are extracted: the research goal, the identified gap, the methods used, and the reported limitations. A comparative discussion is then carried out across the selected papers to assess their contributions and shortcomings. Based on this discussion, the research gap is derived then a research question is formulated to directly address that gap. The research question then serves as the basis for identifying the research objective of this study.

Design

The design phase addresses the design of the proposed solution and is structured around two complementary activities. First, an analysis is conducted to examine existing works that have attempted to address similar or related scientific problems to the research gap identified in the literature review. This analysis yields four core outputs: (1) functional requirements, which define the capabilities the solution must provide; (2) non-functional requirements, which specify the quality attributes and constraints the solution must satisfy; (3) system components, which identify the key building blocks of the proposed solution; and (4) partial solutions, which represent existing approaches that partially address the research gap.

Based on these outputs, the four outputs are used simultaneously to develop the conceptual model of the solution. The model is then described from two perspectives. The behavioral perspective characterizes the dynamic aspects of the solution, including process flows, operational steps, and interaction sequences. The structural perspective characterizes the static architecture of the solution, including system layers and their components.

Evaluation

This phase is dedicated to evaluating the conceptual model. A demonstration is implemented to represent the conceptual model and is subject to both verification and validation to ensure it correctly reflects the proposed conceptual model and fulfills its intended purpose. The Delphi method is adopted as the evaluation strategy, where interviews are conducted with domain experts in an iterative manner. The evaluation is structured around the TOE framework, assessing the conceptual model across three dimensions: Technical, Organizational, and Environmental. Each iteration involves presenting the model and the implemented demonstration, collecting input through expert interviews across these three dimensions, refining the conceptual model, and then updating the demonstration accordingly. This iterative cycle continues until a satisfactory level of consensus and quality is achieved.

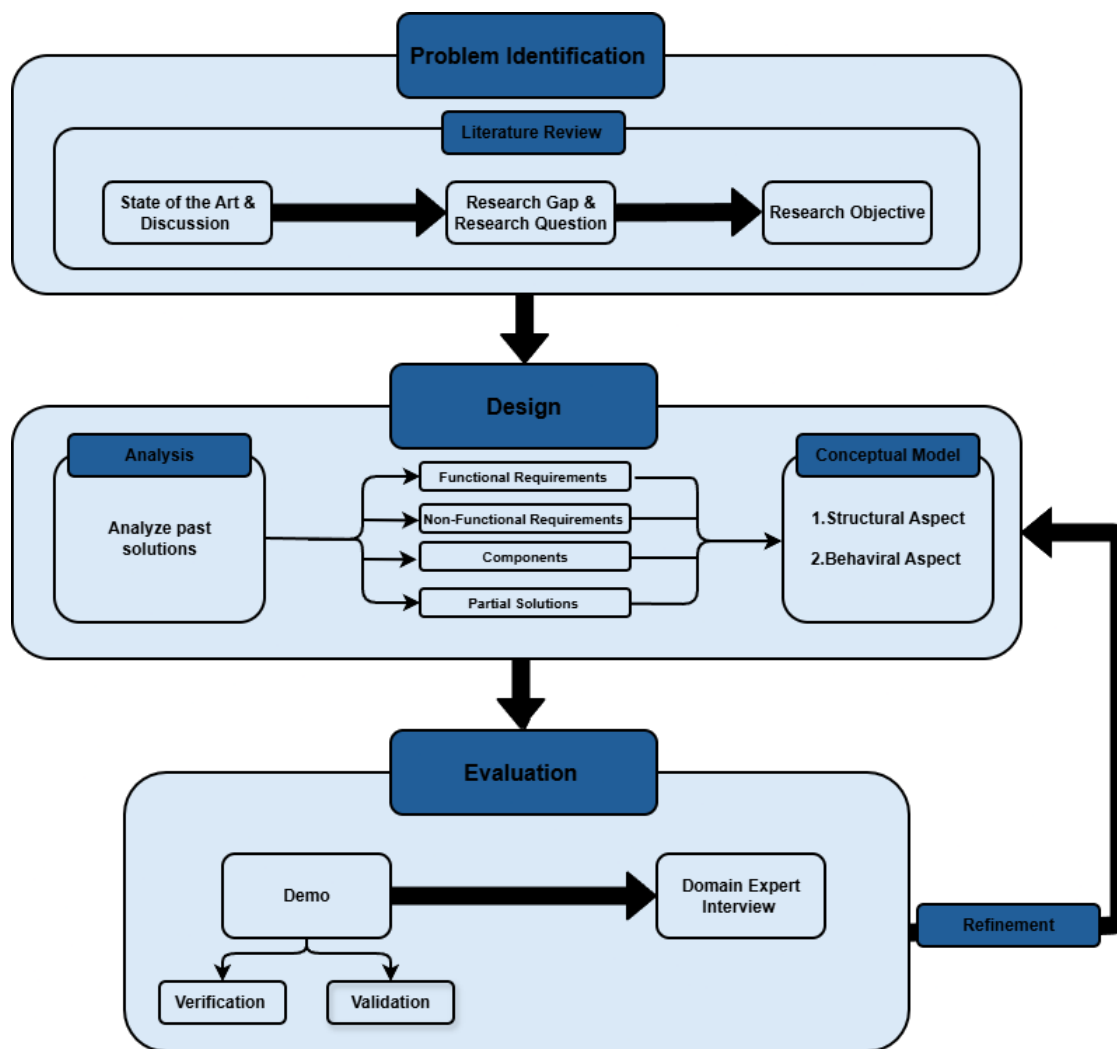


Figure 14 Solution Approach Diagram

7 Implementation

This chapter applies the solution approach presented in the previous chapter, following the same three phases: problem identification, design, and evaluation.

7.1 Problem identification

This phase is already done throughout the literature review in Chapter 5, which analyzed recent studies on AI in financial auditing in the state of the art followed by discussion which is a comparison between the papers. The review shows that AI improves efficiency, fraud detection, and data analysis, but most models focus only on accuracy without considering how results can be explained or justified in real audit work.

The findings also reveal that current research is fragmented and lacks a practical approach for integrating AI into auditing processes. Most AI systems operate as “black boxes,” making their decisions difficult to understand. This contradicts the core requirement of auditing, which depends on transparency, accountability, and clear justification of decisions.

This results in the following **scientific gap: there is still a lack of practical artifacts that show how AI can be used in a transparent and reliable way inside the audit process**. Consequently, this gap gives rise to the following **scientific question: how can Explainable AI (XAI) be integrated into financial auditing to ensure that AI outputs are transparent, understandable, and supportive of auditors' decisions?** As a result, the following **research objective was formulated: to develop a conceptual model for integrating Explainable Artificial Intelligence (XAI) into the financial auditing process, addressing the identified scientific gap, through which audit transparency, interpretability, and reliability can be enhanced to improve the overall quality of AI-supported audit processes**.

Since the identified gap will be addressed through the development of a conceptual model as a scientific artifact, Design Science Research (DSR) is chosen because it is the most suitable methodology for this study, as DSR is specifically designed to guide the construction and evaluation of innovative artifacts that solve real-world problems in a rigorous and structured manner.

7.2 Design

This section is divided into two parts. In the first part, analysis of existing literature is conducted to address solutions for the gap identified in the previous section. Functional requirements, non-functional requirements, partial solutions, and key building blocks are extracted from the reviewed papers. In the second part, a conceptual model is constructed by building upon existing work, with additional innovations introduced. The model will be presented from both a structural and a behavioral perspective.

7.2.1 Analysis

This section analyzes existing literature to propose solutions for the identified gap, extracting functional and non-functional requirements, partial solutions, and key building blocks.

Paper 1: AuditAgent — Expert-Guided Multi-Agent Reasoning for Cross-Document Fraudulent Evidence Discovery (Bai et al., 2025)

AuditAgent is a multi-agent system for detecting financial fraud in statements, using real-world data (FinFraud-Real), Bayesian risk modeling, retrieval of relevant evidence, and multi-expert reasoning to achieve higher recall than standard LLM and agent baselines.

Functional Requirements

Cross-document financial fraud evidence is retrieved and located across multiple fiscal-year reports, while subject-level fraud risk priors are learned from past enforcement data to guide detection. Domain-specific reasoning is then applied at both within-document and cross-document levels through specialized agent modules. Ultimately, structured fraud descriptions are generated, incorporating the related reports, a concise fraud summary, and supporting text evidence.

Non-Functional Requirements

Interpretability is supported through clear evidence chains that link each fraud finding to specific document parts. The system prioritizes recall over precision, as missing fraud signals proves more costly than false positives in auditing. It remains stable across varying document lengths, handling up to about 79,000 tokens, while embedding domain knowledge directly into the reasoning process rather than depending on general agent behavior.

System Components / Building Blocks

The Variational Bayesian Subject Prior Model learns fraud likelihood scores and links between subjects from past cases, effectively reducing the search space. This feeds into the Prior-Guided Hybrid Retriever, which combines sparse keyword matching and dense semantic matching in parallel, focusing on high-risk subjects. Reasoning then advances through the Multi-Expert LLM Reasoning Module, featuring single-document experts, a cross-document expert, and a synthetic expert that synthesizes evidence for a comprehensive fraud assessment. Evaluation draws from the FinFraud-Real Benchmark Dataset, a real-world annotated collection of 1,570 fraud cases sourced from CSRC documents and multi-year financial reports.

Partial Solutions

A partial contribution to the identified gap is offered through the generation of structured fraud descriptions that link each finding to specific document segments, providing a degree of output traceability. Evidence chains are constructed and exposed to the reviewer, which partially addresses the need for interpretable AI outputs. However, no formal XAI mechanism is applied, and findings are not grounded in any professional auditing standard.

Paper 2: HCLA — Human-Centered LLM-Agent System for Detecting Anomalous Digital Asset Transactions (Na et al., 2025)

HCLA is a human-centered multi-agent system for detecting anomalies in crypto transactions, combining query parsing, XGBoost-based detection, and plain-language explanations to achieve strong accuracy while improving interpretability and user trust.

Functional Requirements

Natural language queries are parsed and transformed into structured detection schemas, eliminating the need for technical input. Transaction-level anomaly probabilities are then computed using time-based and transaction features. For each flagged anomaly, explanations are generated that tie the score directly to identifiable behavior patterns. Users can refine results and pursue follow-ups through an intuitive conversational loop.

Non-Functional Requirements

Transparency is ensured by dividing the pipeline into three distinct stages: Parsing, Detection, and Explanation, while accessibility empowers non-experts via natural language interactions through a user-friendly Gradio interface. Modularity allows seamless replacement of the detection model without disrupting the overall flow, and response times stay under two minutes per query to support practical deployment. Trust is validated through a user study that compares user understanding, clarity, and confidence against a basic dashboard.

System Components / Building Blocks

The Parsing Agent converts user queries into structured JSON, capturing essentials like wallet IDs, time ranges, and transaction details. The Detection Agent then calculates per-transaction anomaly probabilities using key features such as frequency, direction, and connections. Following that, the Explanation Agent transforms these scores into clear natural language explanations while handling follow-up questions. A Central LLM Controller orchestrates communication between agents via JSON, ensuring consistency throughout the stages. Users interact seamlessly through the Gradio Web Interface, which powers conversational experience.

Partial Solutions

A partial solution is provided through the explicit separation of the pipeline into three auditable stages: Parsing, Detection, and Explanation ensuring that each step is visible and queryable. Natural language explanations are generated and tied to specific transaction features, which partially address the transparency requirement. Nevertheless, the explanations are produced by general-purpose LLMs without reference to domain-specific auditing standards or regulatory frameworks.

Paper 3: Watson (2025) Explain First, Trust Later: LLM-Augmented Explanations for Graph-Based Crypto Anomaly Detection

A modular, regulation-agnostic crypto fraud detection pipeline combines GNN-based anomaly scoring, explainable AI (Graph LIME), and LLM-generated narratives, with a dashboard for investigations supporting interpretable detection and fraud type classification.

Functional Requirements

Wallet-level anomaly scores emerge from an unsupervised graph neural network trained on blockchain transaction data, with feature-level importance highlighting the transaction attributes that most influenced each flagged anomaly. An LLM then generates human-readable fraud explanations, detailing the fraud type, confidence score, and a curated list of behavioral evidence. This approach advances beyond binary labels through a fraud type classification scheme derived directly from LLM outputs. Investigators can dive deeper via an interactive dashboard that facilitates exploration of flagged wallets and their associated explanations.

Non-Functional Requirements

Model independence allows the GNN, XAI explainer, and LLM to be updated or swapped individually, while regulation independence enables adaptation to diverse legal frameworks without altering the core structure. Reproducibility is bolstered by fixing the LLM temperature at 0.2 and running it five times per node for consistent outputs; notably, the LLM handles only post-detection explanations, preserving reliability in anomaly detection itself. Human review remains integral, as investigators verify flagged cases before any actions proceed.

System Components / Building Blocks

AnomalyGCN, a two-layer graph neural network trained via self-supervised methods, generates anomaly scores for each node. The GraphLIME Explainer then assigns importance weights to features, revealing which ones drove the anomaly. These weights are processed by an LLM Explanation Module alongside node data to produce structured outputs, including fraud explanations, fraud types, confidence levels, and supporting evidence. A consensus mechanism runs the model five times per node with majority voting to enhance output consistency. A Fraud Type Decision Tree, trained on LLM outputs and feature weights, provides interpretable classification rules. All findings are presented through an Interactive Dashboard that visualizes the transaction graph with color-coded fraud labels and detailed node explanations.

Partial Solutions

A more direct partial solution is offered through the integration of GraphLIME feature attributions with LLM-generated narratives, enabling human-readable explanations that are grounded in model behavior. Fraud type classification is also supported, extending interpretability beyond binary detection. However, the pipeline operates within the cryptocurrency domain and is not connected to formal auditing standards or enterprise financial environments.

Discussion

The three works share a common emphasis on explainability and modular design, yet differ in scope and methodological focus. While Bai et al. (2025) targets cross-document financial fraud using expert-guided multi-agent reasoning and Bayesian priors to maximize recall, Na et al. (2025) adopts a human-centered architecture that integrates structured query parsing, traditional machine learning (XGBoost), and conversational explanations to enhance usability and trust. In contrast, Watson (2025) emphasizes a regulation-agnostic, modular pipeline leveraging graph neural networks and post-hoc explainability (GraphLIME) with LLM-generated narratives, extending beyond detection to fraud type classification. Collectively, the papers illustrate a progression from domain-specific, expert-driven detection (Bai et al., 2025), to user-centric interactive systems (Na et al., 2025), and finally to highly modular, explainable architectures that decouple detection and interpretation (Watson, 2025), highlighting trade-offs between performance optimization, accessibility, and system generalizability.

7.2.2 Modelling

In this section, a conceptual model is constructed by building upon existing work, with additional innovations introduced. The functional requirements and non-functional requirements are first represented, then the model is described from both structural and behavioral perspectives.

7.2.2.1 Functional Requirements

Functional requirements define the specific functions and behaviors that a system is required to perform, as outlined in Table 3. Requirements derived from existing work are supported by relevant citations, while requirements introduced as part of the proposed contribution are justified through logical argumentation.

ID	Functional Requirement
FR1	The system shall allow an auditor to upload a General Ledger file and an ERP configuration file through a user interface, triggering the audit pipeline automatically, as a single controlled entry point reduces the risk of inconsistent or incomplete inputs reaching downstream components (Na et al., 2025).
FR2 (Novel Contribution)	The system shall ingest, normalize, and validate the uploaded General Ledger file row by row, producing a clean DataFrame and a structured list of IngestionIssue objects that capture the row index, column name, and a description of each validation failure, as structured error records allow auditors to trace and correct data quality issues before analysis begins.
FR3 (Novel Contribution)	The system shall parse the ERP configuration export to extract controls, users, accounts, and rules, and construct a directed knowledge graph encoding all ERP relationships for use in downstream control attribution, as a graph representation preserves the relational structure of ERP controls and enables precise traversal during attribution.
FR4	The system shall engineer domain specific numerical features from the validated General Ledger data, including time-based patterns, amount ratios, and account level statistics, and normalize them into a feature matrix ready for anomaly detection, as domain specific features capture financial behavior patterns that generic features would miss (Na et al., 2025; Watson, 2025).
FR5	The system shall apply a multi model anomaly detection ensemble comprising Isolation Forest, Local Outlier Factor, and an Autoencoder, rank normalize each model's scores, and flag General Ledger entries whose averaged ensemble score exceeds a configurable threshold (Na et al., 2025).
FR6	The system shall compute SHAP contribution values for each flagged General Ledger entry using a KernelExplainer, producing an F by K matrix where F is the number of flagged entries and K is the number of engineered features, to support feature level interpretability (Watson, 2025).
FR7 (Novel Contribution)	The system shall traverse the ERP control knowledge graph for each flagged entry to identify the specific control failure responsible, such as an SoD Violation or a Missing Approval, and record the full graph traversal path as a ControlAttribution object as this enables auditors to pinpoint exactly which control failed rather than relying on generic anomaly flags.

ID	Functional Requirement
FR8 (Novel Contribution)	The system shall retrieve the most semantically relevant ISA audit standard paragraph for each flagged entry by constructing a query from its SHAP value and attribution type and performing similarity search against a pre indexed ISA corpus stored in a vector database as this grounds each finding in an officially recognized auditing standard, strengthening the report's professional validity.
FR9	The system shall generate a readable prose narrative for each flagged entry by sending its merged SHAP values, ERP control attribution, and retrieved ISA paragraph to a locally hosted language model, which acts solely as a text renderer to format pre-determined findings without independent reasoning (Watson, 2025).
FR10	The system shall present all findings through an interactive dashboard that displays a ranked anomaly list with expandable narrative explanations and shall allow the auditor to export the complete audit report as a PDF or Excel file (Na et al., 2025).

Table 3 Functional Requirements

7.2.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints under which a system operates, as outlined in Table 4. Requirements derived from existing work are supported by relevant citations, while requirements introduced as part of the proposed contribution are justified through logical argumentation.

ID	Non-Functional Requirement
NFR1 (Novel Contribution)	The system shall execute all pipeline stages without transmitting any data to external servers, and the locally hosted language model shall ensure that all audit data always remains within the organization's infrastructure, as financial audit data is highly sensitive and must not be exposed to third party services under any circumstance.

ID	Non-Functional Requirement
NFR2	The system shall provide interpretable outputs for every flagged entry by combining SHAP feature level contributions, ERP graph traversal paths, and ISA paragraph citations, ensuring that each finding can be traced back to its source evidence (Bai et al., 2025; Watson, 2025).
NFR3	The anomaly detection ensemble shall be designed to prioritize recall over precision, minimizing the risk of undetected fraudulent or erroneous General Ledger entries even at the cost of a higher false positive rate, given the high stakes of missed findings in financial auditing (Bai et al., 2025).
NFR4	The system shall be modular such that any individual component, including the anomaly detection models, the SHAP explainer, the ERP parser, and the language model, can be replaced or upgraded independently without disrupting the overall pipeline (Na et al., 2025; Watson, 2025).
NFR5	The language model's role shall be strictly limited to formatting pre-determined SHAP values, control attributions, and ISA citations into prose, with output length capped by a configurable character limit and generation time bounded by a timeout parameter, in order to minimize hallucination risk (Watson, 2025).
NFR6 (Novel Contribution)	The system shall handle the alternative pipeline branch gracefully such that when the anomaly detection ensemble returns zero flagged entries the orchestrator shall call the dashboard's render no findings method and terminate the pipeline cleanly without invoking explanation or narrative generation components, as invoking downstream components unnecessarily when no anomalies exist wastes computational resources and may mislead auditors.
NFR7	The ISA vector index must be pre-indexed with a consistent embedding model and similarity metric. Its configuration should be version-controlled to ensure reproducible and comparable retrieval results across audit runs, preserving audit reliability (Watson, 2025).

Table 4 Non-Functional Requirements

In this section, the system is presented from a structural perspective using a UML class diagram. Figure 15 illustrates the main classes, their attributes, methods, and relationships between them.



LangGraphOrchestrator

The LangGraphOrchestrator is the central coordinator of the entire pipeline. It exposes a single method `start_pipeline(gl_file, erp_config_file)` which serves as the entry point for the whole system. It receives the two uploaded files and sequentially drives every downstream component, returning a PipelineState object representing the completed audit run.

GLIngestionAgent

This component is responsible for reading and validating the uploaded General Ledger file. It holds `column_mapping` which is a dictionary that maps raw column names to internal standardized names. Its `ingest()` method reads the Excel file, normalizes the columns, validates each row, and returns the clean DataFrame and a list of IngestionIssue objects for any invalid rows found. The `validate_row()` method is an internal helper that checks each individual row and returns a Boolean.

IngestionIssue

This is a pure data class with no methods; it captures the details of a single GL row validation error. It stores three attributes: `row_index` of the failing row, the column that caused the failure, and an `issue_description` providing a human-readable explanation of the problem.

ERPConfigParser & KnowledgeGraphBuilder

This component parses the ERP configuration export and builds the internal control knowledge graph. Its `graph_stats` attribute stores the resulting graph. Its `parse()` method reads the ERP export file, extracts controls, users, accounts, and SoD rules, then internally calls `build_graph()` to construct a directed graph (DiGraph) encoding all ERP relationships. The `parse()` method returns a tuple containing the list of ControlRecord objects and the fully constructed DiGraph. If any entry in the config file is unrecognizable, it produces a ConfigParseIssue object.

ConfigParseIssue

A simple data class that captures a single ERP configuration parsing error. It stores the `record_type` of the entry that failed (for example `SoD_Rule` or `ApprovalWorkflow`) and an `issue_description` explaining what went wrong during parsing.

FeatureEngineeringComponent

This component transforms the validated GL DataFrame into a numerical feature matrix ready for anomaly detection. It holds a StandardScaler instance used to normalize all features to zero mean and unit variance, and a feature_names list naming all engineered features (also used by SHAP for labeling). Its single method compute_features() applies domain-specific calculations such as time-based patterns, amount ratios, and account-level statistics, then returns the resulting ndarray feature matrix.

AnomalyDetectionEnsemble

This component runs the multi-model anomaly detection at the core of the system. Its attributes configure each sub-model: threshold sets the minimum ensemble score required to flag an entry, if_estimators controls the Isolation Forest tree count, lof_neighbors sets the neighbor count for Local Outlier Factor, and ae_architecture defines the Autoencoder layer sizes. The detect() method runs all three models on the feature matrix, rank-normalizes each model's scores, averages them into a single ensemble score per entry, and returns a DataFrame of entries that exceeded the threshold which are the flagged rows.

SHAPEngine

The SHAPEngine computes feature-level explanations for each flagged entry using SHAP (SHapley Additive exPlanations). Its background_samples attribute controls how many samples the KernelExplainer uses for approximation, nsamples sets the per-row sample count, and timeout caps the maximum computation time. The explain() method takes the flagged rows and the full feature matrix, runs KernelExplainer, and returns shap_column_names: List; a collection of column names where each entry represents the top contributing feature (highest SHAP value) that led the detectors to flag each row.

ERPControlAttributionAgent

This component traces which specific ERP control failed for each flagged GL entry. Its attribute() method takes the flagged rows and the ERP control graph (DiGraph), traverses the graph per flagged entry to identify the matched control attribute that allowed the entry through, and produces one ControlAttribution object per flagged row.

ControlAttribution

This data class represents the specific ERP control failure identified for one GL entry. It stores the `gl_row_id` of the flagged entry, the `attribution_type` categorizing the failure (for example `SoD_Violation` or `Missing_Approval`), and the `traversal_path` which is the sequence of graph nodes visited to reach this conclusion.

ExplanationMergeComponent

This component merges the outputs of the SHAPEngine and the ERPControlAttributionAgent into a single unified explanation per entry. It has no persistent attributes. Its `merge()` method takes the list of columns names from SHAPEngine and the list of control attributions, pairs each flagged GL row with its corresponding `shap_column_name` and its `ControlAttribution` object, and produces one `MergedExplanation` per entry.

MergedExplanation

A data class holding the combined explanation for one flagged entry. Its attributes are the `gl_row_id`, the `ensemble_score` from the detection step, `shap_column_name`, and `erp_attribution` referencing the associated `ControlAttribution` object identifying the specific ERP control failure.

ISARAGRetrieval

This component retrieves relevant ISA paragraphs for each merged explanation using vector similarity search. The `retrieve()` method takes each `MergedExplanation`, internally constructs a query from its `shap_column_name` and `erp_attribution.attribution_type`, uses that query to search the pre-chunked and indexed ISA corpus from the vector database by similarity search, and returns a list of `EnrichedExplanation` objects with the matching ISA paragraphs.

EnrichedExplanation

A data class representing the complete three-layer explanation for one flagged entry. It holds the `gl_row_id`; an `isa_paragraph` string containing the retrieved ISA; a `shap_column_name` carrying the SHAP contribution value for the engineered features of the flagged entry; an `erp_attribution` reference to the associated `ControlAttribution` object identifying the specific ERP control failure and traversal path.

LocalLLM

This component converts enriched explanations into readable prose using a locally hosted language model. The `llm_model` attribute specifies which locally hosted language model is used, ensuring no data leaves the machine; `max_chars` caps narrative length; and `timeout` limits generation time. The `generate()` method sends each `EnrichedExplanation` to the language model as a structured prompt. The language model acts only as a text renderer, meaning it formats pre-determined SHAP values, control attributions, and ISA citations into prose without performing any independent reasoning, which minimizes hallucination risk. It writes the generated prose directly into the `list[final_outputs]` (list of strings containing the LLM text for each row) using `EnrichedExplanation` and returns the full list which contains LLM generated text for each row.

ISAVectorIndex

This is a passive datastore with no methods; it holds the pre-indexed ISA audit standard corpus used by `ISARAGRetrieval` at retrieval time. It stores three attributes: `embedding_model` specifying the model used to encode ISA paragraphs during indexing, `similarity_metric` defining the distance function used for nearest-neighbor search, and `index_name` identifying the target index within the vector database.

UI_Dashboard

The `UI_Dashboard` is the auditor-facing UI component that presents all findings interactively. Its `render_findings(list[final_outputs])` method displays a ranked anomaly Data frame where each row can be expanded to reveal the LLM generated text. Its `render_no_findings()` method is called by the Orchestrator when the anomaly detection ensemble returns zero flagged entries ($F = 0$) and displays a “No entries exceed threshold” message to the auditor, ending the pipeline cleanly without proceeding to explanation or narrative generation. Its `export()` method allows the auditor to download all findings as a PDF or Excel file for audit documentation.

7.2.2.4 Behavioral Aspect

In this section, the system is presented from a behavioral perspective using a sequence diagram. Figure 16 highlights the flow of messages and the order of operations during execution.

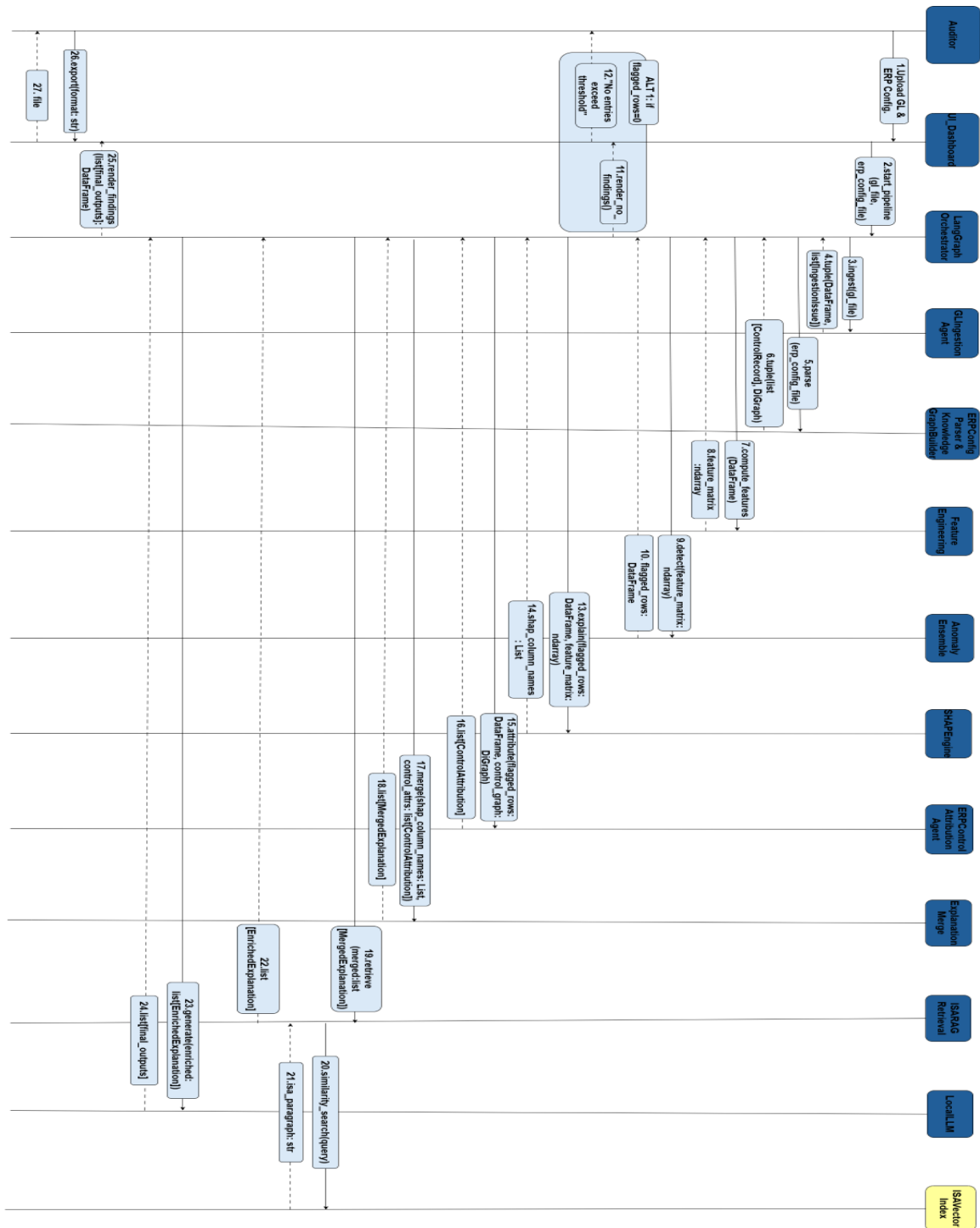


Figure 16 Conceptual Model Sequence Diagram

Step 1: Auditor Uploads GL and ERP Configuration Files

The process begins when the Auditor interacts with the UI_Dashboard by uploading two input files: the General Ledger (GL) file and the ERP configuration file. This action initiates the entire audit pipeline.

Step 2: UI Dashboard Triggers the Orchestrator

Upon receiving the uploaded files, the UI_Dashboard calls the `start_pipeline(gl_file, erp_config_file)` method on the LangGraphOrchestrator. This passes both files to the central coordinator, which is responsible for driving all subsequent pipeline stages.

Step 3: GL File Is Sent to the Ingestion Agent

The LangGraphOrchestrator calls the `ingest(gl_file)` method on the GLIngestionAgent, passing the uploaded GL file for processing. This initiates the data validation and normalization phase.

Step 4: Ingestion Agent Returns the Cleaned DataFrame and Issues

The GLIngestionAgent reads the Excel file, normalizes column names using its `column_mapping` dictionary, and validates each row using the internal `validate_row()` helper. It returns a clean DataFrame and a list of IngestionIssue objects, each capturing any row-level validation failures through its `row_index`, `column`, and `issue_description` attributes.

Step 5: ERP Configuration File Is Sent to the Config Parser

LangGraphOrchestrator calls the `parse(erp_config_file)` method on the ERPConfigParser & KnowledgeGraphBuilder, passing the ERP configuration export for processing.

Step 6: Config Parser Returns Control Records and the Knowledge Graph

The ERPConfigParser & KnowledgeGraphBuilder reads the ERP export and extracts controls, users, accounts, and Segregation of Duties (SoD) rules. It internally calls `build_graph()` to construct a directed graph (DiGraph) encoding all ERP relationships. The method returns a tuple containing the list of ControlRecord objects and the fully constructed DiGraph. Any unrecognizable entry produces a ConfigParseIssue object carrying the `record_type` and `issue_description`.

Step 7: Feature Engineering Is Initiated on the Validated DataFrame

The LangGraphOrchestrator calls `compute_features(gl: DataFrame)` on the FeatureEngineeringComponent, passing the validated GL DataFrame produced in Step 4.

Step 8: Feature Matrix Is Returned to the Orchestrator

The FeatureEngineeringComponent applies domain-specific calculations, including time-based patterns, amount ratios, and account-level statistics, and normalizes all features using its internal scaler (a StandardScaler instance). It returns a numerical ndarray feature matrix, where each column corresponds to an entry in the `feature_names` list.

Step 9: Anomaly Detection Is Triggered on the Feature Matrix

The LangGraphOrchestrator calls `detect(feature_matrix: ndarray)` on the AnomalyDetectionEnsemble, passing the feature matrix produced in Step 8.

Step 10: Flagged Rows Are Returned as a DataFrame

The AnomalyDetectionEnsemble runs three sub-models, namely Isolation Forest, Local Outlier Factor, and an Autoencoder, on the feature matrix. It rank-normalizes each model's scores and averages them into a single ensemble score per entry. Any entry whose ensemble score exceeds the configured threshold is included in the returned `flagged_rows: DataFrame`.

ALT Branch: If `flagged_rows = 0` (No Anomalies Detected)

If the AnomalyDetectionEnsemble returns zero flagged entries (i.e., $F = 0$), the pipeline enters an alternative branch which includes step 11,12 otherwise the pipeline goes to step 13.

Step 11: Orchestrator Calls `render_no_findings()`

The LangGraphOrchestrator calls `render_no_findings()` on the UI_Dashboard to signal that no anomalies were detected.

Step 12: Dashboard Displays the No-Findings Message

The UI_Dashboard displays the message “No entries exceed threshold” to the Auditor. The pipeline terminates cleanly at this point without proceeding to explanation or narrative generation.

Step 13: SHAP Explanations Are Requested for Flagged Rows

When flagged rows exist ($F > 0$), the LangGraphOrchestrator calls `explain(flagged_rows: DataFrame, feature_matrix: ndarray)` on the SHAPEngine, passing both the flagged entries and the full feature matrix.

Step 14: SHAP Values Matrix Is Returned

The SHAPEngine uses a KernelExplainer, configured by its `background_samples`, `nsamples`, and `timeout` attributes, to compute SHAP contribution values for each flagged entry. It returns `shap_column_names: List` which is a collection of column names where each entry represents the top contributing feature (highest SHAP value) that led the detectors to flag each row.

Step 15: ERP Control Attribution Is Requested

LangGraphOrchestrator calls `attribute(flagged_rows: DataFrame, control_graph: DiGraph)` on the ERPControlAttributionAgent, passing the flagged entries alongside the ERP knowledge graph built in Step 6.

Step 16: List of ControlAttribution Objects Is Returned

The ERPControlAttributionAgent traverses the DiGraph for each flagged entry to identify the specific ERP control that failed to prevent the anomalous transaction. It returns one ControlAttribution object per flagged row, each storing the `gl_row_id`, the `attribution_type` (for example, `SoD_Violation` or `Missing_Approval`), and the `traversal_path`, which is the sequence of graph nodes visited to reach the conclusion.

Step 17: Explanation Merge Is Requested

The LangGraphOrchestrator calls `merge(shap_column_names: List, control_attrs: list[ControlAttribution])` on the ExplanationMergeComponent, passing both the SHAP column names list from Step 14 and the control attribution list from Step 16.

Step 18: List of MergedExplanation Objects Is Returned

The ExplanationMergeComponent pairs each flagged GL row with its top-1 SHAP contribution value and its corresponding ControlAttribution object. It produces one MergedExplanation per entry, containing the `gl_row_id`, the `ensemble_score`, the `shap_column_name`, and the `erp_attribution` reference.

Step 19: ISA RAG Retrieval Is Triggered

The LangGraphOrchestrator calls `retrieve(merged: list[MergedExplanation])` on the ISARAGRetrieval component, passing the full list of merged explanations.

Step 20: Similarity Search Is Performed on the ISA Vector Index

For each MergedExplanation, the ISARAGRetrieval component internally constructs a query from the entry's `shap_column_name` and `attribution_type`. The system issues two separate queries to the ISAVectorIndex using `similarity_search(query)`: one retrieves the top-1 ISA paragraph most similar to `shap_column_name`, and the other retrieves the top-1 ISA paragraph most similar to `attribution_type`, which is a passive datastore holding the pre-chunked and indexed ISA audit standard corpus, configured with an `embedding_model`, a `similarity_metric`, and an `index_name`.

Step 21: ISA Paragraph Is Returned from the Vector Index

The ISAVectorIndex returns the most relevant `isa_paragraph: str` for each query, representing the ISA audit standard paragraph most semantically similar to the flagged entry's characteristics.

Step 22: List of EnrichedExplanation Objects Is Returned

The ISARAGRetrieval component assembles the results into a list of EnrichedExplanation objects, each holding the `gl_row_id`, the retrieved `isa_paragraph`, the `shap_column_name`, and the `erp_attribution` reference. This list is returned to the LangGraphOrchestrator.

Step 23: Narrative Generation Is Triggered

The LangGraphOrchestrator calls `generate(enriched: list[EnrichedExplanation])` on the LocalLLM component, passing all enriched explanations.

Step 24: LLM Returns the Final Outputs

The LocalLLM sends each EnrichedExplanation to the locally hosted language model (identified by its llm_model attribute) as a structured prompt. The model acts solely as a text renderer; it formats the pre-determined shap_column_name, control attributions, and ISA paragraphs into readable prose without performing independent reasoning, which minimizes hallucination risk. Output length is capped by max_chars and generation time by timeout. The method returns final_outputs[str]: a list of LLM-generated narrative strings, one per flagged row.

Step 25: Dashboard Is Called to Render Findings

The LangGraphOrchestrator calls render_findings(list[final_outputs]: DataFrame) on the UI_Dashboard, passing the final narrative outputs for display.

Step 26: Auditor Requests an Export

The Auditor interacts with the UI_Dashboard by calling export(format: str) to download the findings in a preferred format, either PDF or Excel, for audit documentation purposes.

Step 27: Export File Is Delivered to the Auditor

The UI_Dashboard returns the generated file to the Auditor, completing the pipeline. The auditor now has both an interactive view of ranked anomalies and a downloadable audit report.

7.3 Evaluation

This section presents the evaluation of the proposed system conceptual model. The section is structured as follows. First, a demonstration is introduced, which implements a selected subset of the full conceptual pipeline in order to make the system's core contribution observable and testable then a Technical Stack, Installation Process and demonstration Mapping are Explained. The prototype is then assessed through technical verification, which checks whether each component produces the expected output, and validation, which confirms that the requirements defined in Section 7.2.2 are satisfied. Following this, an expert interview is conducted with domain practitioners using the Delphi method, an iterative expert evaluation strategy, structured around the TOE (Technological, Organizational, and Environmental) framework. If shortcomings are identified, the conceptual model is revised and any resulting changes to the prototype are addressed in the refinement step that follows.

7.3.1 Demonstration Introduction

The full conceptual model described in Section 7.2 encompasses a twenty-seven-step end-to-end pipeline, beginning with the upload of a General Ledger (GL) file and ERP configuration export and concluding with the delivery of a downloadable audit report to the auditor. Implementing the complete pipeline within the scope of this thesis would require live access to an ERP system and a labelled GL dataset, which were not available. A focused prototype was therefore constructed that targets the most technically novel contribution of this work, specifically the integration of Retrieval-Augmented Generation with local large language model inference for standards-compliant audit narrative generation, covering steps 19 to 25 of the conceptual sequence diagram (Figure 16).

To faithfully simulate the handoff boundary at step 18, rather than receiving a `MergedExplanation` object produced by the `ExplanationMergeComponent`, the prototype accepts equivalent inputs directly from the auditor through a web interface, where an ERP control attribute and a SHAP feature column name are selected from curated dropdown menus. These two inputs represent precisely the information that the `ExplanationMergeComponent` would supply in the full system. From this point onward, all remaining pipeline steps are executed through the `LangGraphOrchestrator`: ISA standard retrieval via semantic search (steps 19–22), narrative generation via a local language model (steps 23–24), and results rendering to the auditor (step 25). The export step (steps 26–27) is omitted as it falls outside the scope of the core AI contribution.

The following simplifications apply throughout this demonstration and are stated here once to keep each subsequent step focused on the demonstrated behavior only. In the full conceptual model, each `MergedExplanation` carries a `gl_row_id`, an `ensemble_score`, a `shap_column_name`, and an `erp_attribution` that includes a `traversal_path` from the ERP knowledge graph, with `render_findings()` outputting a complete `DataFrame` of row-level narratives. In the demonstration, `gl_row_id` and `ensemble_score` are removed as only one pre-flagged row is simulated, `traversal_path` is removed as no ERP graph traversal is performed, and the output is a single narrative string rather than a full `DataFrame`. The prototype is therefore not a simplified version of the system; it is a complete and fully operational realization of the system's most technically novel sub-pipeline.

7.3.2 Technical Stack and Dependencies

This subsection presents the key technologies used to build the prototype, describing the role and justification of each tool within the system architecture.

The prototype was implemented in Python and relies on the following technologies:

Streamlit (≥ 1.28) serves as the web application framework. It enables rapid construction of an interactive single-page UI without requiring a separate front-end codebase. The `st.cache_resource` decorator ensures that the embedding model, vector index, and language model are loaded once per server session and reused across requests, eliminating repeated initialization overhead.

ChromaDB (≥ 0.4) is the persistent vector database used to store paragraph-level embeddings of the ISA standard corpus. ChromaDB's `PersistentClient` writes the index to disk under `chroma_db/`, ensuring that the index survives application restarts without requiring re-embedding. The collection is configured with cosine distance (`hnsw:space: cosine`), which is the appropriate metric for normalized dense embeddings.

Sentence-Transformers (≥ 2.2) with BAAI/bge-large-en-v1.5 provides the embedding capability through two components: the Sentence-Transformers library, which is a Python framework that provides a simple interface to load and run embedding models, and BAAI/bge-large-en-v1.5, which is the actual model loaded by that library to produce the vector representations. BGE-large is a 335-million-parameter encoder model trained using contrastive learning, meaning it learns to place semantically similar texts close together in the vector space and dissimilar texts far apart. This makes it well-suited for retrieval tasks where the goal is to find the most relevant passage for a given query. The model is used in two places in the prototype: during indexing, it converts each ISA paragraph into a dense numerical vector that is stored in ChromaDB; at query time, it converts the user's input into a vector of the same space so that a similarity comparison can be made.

pdfplumber (≥ 0.10) extracts raw text from the ISA standard PDFs page by page. It handles the complex layout of IAASB documents, including footnotes and multi-column sections, more reliably than alternatives such as PyMuPDF for text-only extraction.

PyTorch (≥ 2.0) underpins the sentence-transformer inference. When a CUDA-capable GPU is available, all embedding and inference operations are moved to the GPU automatically; otherwise, the prototype falls back to CPU without any code change.

Ollama is the local LLM serving runtime. It runs as a background process on the user's machine and exposes an HTTP API at a localhost. The prototype communicates with it via standard HTTP requests, meaning no additional Python SDK is required. The default model is llama3.1:8b-instruct-q4_K_M, a 4-bit quantized instruction-tuned variant of Meta's Llama 3.1 8B that runs on consumer hardware with approximately 6 GB of VRAM (or RAM when GPU is unavailable).

LangGraph (≥ 0.2) is the graph-based orchestration framework that realizes the LangGraphOrchestrator component from the class diagram (Figure 15). It provides a StateGraph abstraction in which each pipeline step is defined as a node function that reads from and writes to a shared typed state object (PipelineState). The compiled graph is invoked once per auditor request and executes the retrieval, enrichment, and generation nodes sequentially. Using LangGraph makes the pipeline structure explicit, auditable, and extensible: additional nodes for upstream steps can be inserted into the graph without changing the existing node logic.

Requests (standard release) is the HTTP client Library used to communicate with the Ollamaruntime. The prototype posts each prompt to the Ollama API endpoint at <http://localhost:11434/api/generate> and reads the JSON response. Although Ollama requires non dedicated Python SDK, requests must be installed explicitly because it is not part of the Python standard Library.

scikit-Learn (≥ 1.3) and matplotlib (≥ 3.7) support the optional retrieval-evaluation script (src/evaluate_retrieval.py). scikit-learn provides confusion_matrix and classification_report to measure paragraph-level retrieval accuracy against a labelled ground-truth set. matplotlib renders the resulting confusion matrices as PNG figures. These two libraries are not required to run the main application; they are only needed when executing the evaluation script.

7.3.3 Installation Process

This subsection provides a step-by-step guide to setting up and running the prototype on a local machine, covering all required software, model downloads, and launch instructions.

Step 1: Install Ollama and pull the model

Download and install Ollama from the official distribution. Then pull the required model, The ollama serve command must remain running in the background throughout the session.

```
ollama pull llama3.1:8b-instruct-q4_K_M  
ollama serve
```

Step 2: Install Python dependencies

It is recommended to create a virtual environment first. PyTorch must be installed from the PyTorch package index to obtain the correct CUDA build:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121  
pip install streamlit chromadb sentence-transformers pdfplumber langgraph requests scikit-learn matplotlib
```

For CPU-only environments, PyTorch can be installed from the default PyPI index:

```
pip install torch torchvision  
pip install streamlit chromadb sentence-transformers pdfplumber langgraph requests scikit-learn matplotlib
```

Step 3: Place ISA PDF files

Copy the following five IAASB standard PDFs into the project src/Data/ directory:

- ISA 240.pdf
- ISA 265.pdf
- ISA 315.pdf
- ISA 330.pdf
- ISA 530.pdf

Step 4: Launch the application

On first launch, the indexer automatically extracts paragraph-level chunks from all five PDFs, embeds them using BAAI/bge-large-en-v1.5, and stores them in ChromaDB. This one-time process takes approximately two to five minutes depending on hardware. Subsequent launches skip re-indexing and load the persisted collection directly.

```
streamlit run src/app.py --server.port 8501
```

7.3.4 Demonstration Mapping

This sub-section presents a step-by-step walkthrough of the prototype, mapping each executed action to the corresponding step in the sequence diagram (Figure 16). The demonstration covers the ISA corpus preparation phase, the LangGraph pipeline construction, and sequence steps 19 through 25.

Before the main pipeline can execute, three pre-pipeline steps are necessary to establish the execution environment. Pre-Pipeline Step A handles the chunking of ISA standard PDFs into retrievable paragraph-level units, as the retrieval stage depends on semantically granular text segments rather than full documents. Pre-Pipeline Step B handles the embedding and persistent indexing of these chunks into the vector database, as similarity search cannot be performed without a pre-built index. Pre-Pipeline Step C defines the shared typed data structures and pipeline state object that all downstream components read from and write to, as the LangGraph orchestrator requires a consistent state schema to thread data across nodes. These three steps are executed once before the pipeline is invoked and are prerequisites for the correct functioning of all subsequent stages.

Pre-Pipeline Step A

Before the pipeline can retrieve any ISA standard, the ISA text corpus must be processed, split into retrievable units, and stored in the ISAVectorIndex. The first sub-step is chunking as illustrated in Figure 17, the `chunk_isa_document` function implements paragraph-aware chunking over ISA PDF documents, accepting a file path and returning a list of dictionaries each representing a discrete ISA paragraph chunk.

Three state variables are initialized on lines 35–37: `chunks` as the output accumulator, `current_chunk_lines` as a line buffer for the paragraph in progress, and `current_para_id` as the active paragraph identifier. The regular expression compiled on line 38 matches two ISA-specific

identifier formats, numeric such as 33. and annotated such as A42., serving as the sole boundary detection mechanism without reliance on fixed layouts or whitespace heuristics.

The outer loop on lines 40–41 iterates over PDF pages via `pdfplumber`, with line 44 skipping empty pages. The inner loop on line 45 processes each newline-split line; when the regex matches on line 47, lines 49–54 flush the previously buffered paragraph into chunks as a dictionary carrying `para_id`, `text`, and `source`, then lines 55–56 reset the state for the incoming paragraph. Non-matching lines on line 59 are simply appended to the active buffer. The terminal flush on lines 61–66 commits the final paragraph, which would otherwise be lost as no subsequent boundary triggers its append. The function returns the fully populated chunks list on line 67, ensuring each entry corresponds to exactly one ISA paragraph unit and preserving the semantic granularity required for accurate similarity search at retrieval time.

```
34 def chunk_isa_document(pdf_path: str) -> list[dict]:
35     chunks = []
36     current_chunk_lines = []
37     current_para_id = None
38     para_pattern = re.compile(r'^(\d+\.|A\d+\.)\s')
39
40     with pdfplumber.open(pdf_path) as pdf:
41         for page in pdf.pages:
42             text = page.extract_text()
43             if not text:
44                 continue
45             for line in text.split("\n"):
46                 stripped = line.strip()
47                 match = para_pattern.match(stripped)
48                 if match:
49                     if current_chunk_lines and current_para_id:
50                         chunks.append({
51                             "para_id": current_para_id,
52                             "text": " ".join(current_chunk_lines).strip(),
53                             "source": pdf_path
54                         })
55                     current_para_id = match.group(1)
56                     current_chunk_lines = [stripped]
57                 else:
58                     if current_chunk_lines:
59                         current_chunk_lines.append(stripped)
60
61             if current_chunk_lines and current_para_id:
62                 chunks.append({
63                     "para_id": current_para_id,
64                     "text": " ".join(current_chunk_lines).strip(),
65                     "source": pdf_path
66                 })
67     return chunks
68
```

Figure 17 Chunking Process Code

Pre-Pipeline Step B

As illustrated in Figure 18, this code defines the ISAVectorIndex class and the index_isa_corpus function, together implementing the embedding and persistent indexing phase of the ISA corpus preparation pipeline which should be done after chunking step.

The ISAVectorIndex class declares three class-level attributes on lines 73–75: embedding_model set to BAAI/bge-large-en-v1.5, similarity_metric set to cosine, and index_name set to isa_corpus, collectively encoding the retrieval configuration as defined in the conceptual model. The __init__ method on lines 77–83 instantiates a PersistentClient pointing to a local chroma_db directory on line 78, loads the sentence transformer model on line 79, and creates or retrieves the ChromaDB collection on lines 80–82, passing the index name and similarity metric via the hnsw:space metadata key so that cosine distance governs all subsequent nearest-neighbor queries.

The similarity_search method on lines 85–94 encodes an incoming query string into a vector on line 86, issues a single-result query against the collection on lines 87–90, and returns the top retrieved document string on line 92 if present, falling back to an empty string on line 94 to prevent downstream failures.

The standalone index_isa_corpus function on lines 96–113 orchestrates bulk ingestion. Lines 97–99 aggregate all paragraph chunks across the provided PDF paths by invoking chunk_isa_document for each file. Lines 101–104 extract the text bodies and encode them in a single batched pass with batch_size=32. Lines 105–112 persist the resulting embeddings into the ChromaDB collection alongside composite identifiers and metadata carrying source and para_id per chunk, ensuring each stored vector remains traceable to its originating ISA paragraph and document.

```

72 class ISAVectorIndex:
73     embedding_model: str = "BAAI/bge-large-en-v1.5"
74     similarity_metric: str = "cosine"
75     index_name: str = "isa_corpus"
76
77     def __init__(self):
78         self._client = chromadb.PersistentClient(path="chroma_db")
79         self._encoder = SentenceTransformer(self.embedding_model)
80         self._collection = self._client.get_or_create_collection(
81             name=self.index_name,
82             metadata={"hnsw:space": self.similarity_metric}
83         )
84
85     def similarity_search(self, query: str) -> str:
86         query_vector = self._encoder.encode(query).tolist()
87         results = self._collection.query(
88             query_embeddings=query_vector,
89             n_results=1
90         )
91         if results["documents"] and results["documents"][0]:
92             return results["documents"][0][0]
93         return ""
94
95
96 def index_isa_corpus(pdf_paths: list[str], isa_index: ISAVectorIndex) -> None:
97     all_chunks = []
98     for pdf_path in pdf_paths:
99         all_chunks.extend(chunk_isa_document(pdf_path))
100
101     texts = [chunk["text"] for chunk in all_chunks]
102     embeddings = isa_index._encoder.encode(
103         texts, batch_size=32, show_progress_bar=True
104     )
105     isa_index._collection.add(
106         documents=texts,
107         embeddings=embeddings.tolist(),
108         ids=[f"{chunk['source']}_{i}" for i, chunk in enumerate(all_chunks)],
109         metadatas=[
110             {"source": chunk["source"], "para_id": chunk["para_id"]}
111             for chunk in all_chunks
112         ]
113     )
114
115

```

Figure 18 Embedding and Indexing Process Code

Pre-Pipeline Step C

As illustrated in Figure 19, this code defines the typed data structures that form the shared state backbone of the demonstration pipeline, directly reflecting the data class definitions specified in the conceptual class diagram.

The `ControlAttribution` dataclass on line 123 represents the ERP control attribution output, carrying `attribution_type`. The `MergedExplanation` dataclass on lines 130–134 models the handoff object from the `ExplanationMergeComponent`, combining the top SHAP feature name, and a nested `ControlAttribution` instance into a single structured unit. The `EnrichedExplanation` dataclass on line 138 extends this by adding the `isa_paragraph` field to carry the ISA retrieved text, with `erp_attribution`.

The `PipelineState` class on line 145 inherits from `TypedDict` and serves as the shared state object threaded across all `LangGraph` nodes. It holds the two auditor-provided inputs on lines 146–147, the two ISA retrieval results on lines 148–149, and the downstream products on lines 150–151 typed as `Optional` to reflect their conditional population as the pipeline progresses.

```
122 @dataclass
123 class ControlAttribution:
124     attribution_type: str = ""
125
126
127
128
129 @dataclass
130 class MergedExplanation:
131     shap_column_name: str
132     erp_attribution: ControlAttribution
133
134
135
136
137 @dataclass
138 class EnrichedExplanation:
139     isa_paragraph: str = ""
140     shap_column_name: str = ""
141     erp_attribution: ControlAttribution = field(default_factory=ControlAttribution)
142
143
144
145 class PipelineState(TypedDict):
146     shap_column_name: str
147     erp_attribution_type: str
148     isa_paragraph_shap: str
149     isa_paragraph_erp: str
150     enriched_explanation: Optional[EnrichedExplanation]
151     final_output: Optional[str]
152
```

Figure 19 Demonstration Components Code

As illustrated in Figure 20, the LangGraphOrchestrator class implements the central pipeline coordinator as a compiled LangGraph StateGraph, directly realizing the component of the same name defined in the conceptual class diagram.

The `__init__` method on lines 276–280 instantiates all pipeline dependencies through constructor injection, binding ISARAGRetrieval, LocalLLM, and UI_Dashboard as instance attributes before fixing the execution structure via `_build_graph()` on line 280. The `_build_graph` method on lines 282–289 constructs a StateGraph over PipelineState, registers the two node callables on lines 284–285, sets `retrieve_isa` as the entry point on line 286, and connects both nodes sequentially with a directed edge terminating at END on lines 287–288, returning the compiled graph on line 289.

The `start_pipeline` method on lines 291–304 serves as the sole public entry point, constructing the initial PipelineState on lines 294–301 with the two auditor-provided inputs populated and all downstream fields set to empty strings or None. Line 302 invokes the compiled graph to thread state through each node sequentially, after which `render_findings` is called on line 303 outside the graph boundary to preserve correct output ordering, before the final state is returned on line 304.

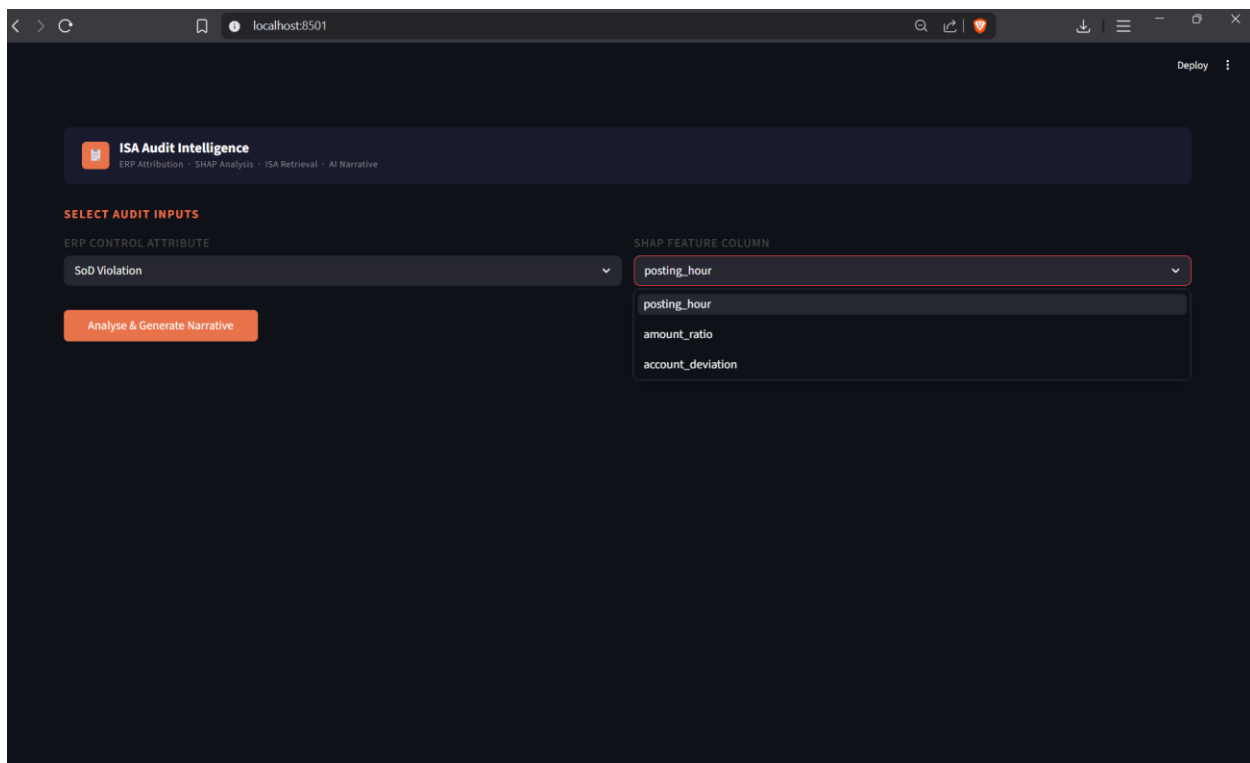
```
275 class LangGraphOrchestrator:
276     def __init__(self, isa_index: ISAVectorIndex):
277         self.isa_retrieval = ISARAGRetrieval(isa_index)
278         self.local_llm = LocalLLM()
279         self.ui_dashboard = UI_Dashboard()
280         self.graph = self._build_graph()
281
282     def _build_graph(self):
283         graph = StateGraph(PipelineState)
284         graph.add_node("retrieve_isa", self.isa_retrieval.retrieve_node)
285         graph.add_node("generate_narrative", self.local_llm.generate_node)
286         graph.set_entry_point("retrieve_isa")
287         graph.add_edge("retrieve_isa", "generate_narrative")
288         graph.add_edge("generate_narrative", END)
289         return graph.compile()
290
291     def start_pipeline(
292         self, shap_column_name: str, erp_attribution_type: str
293     ) -> PipelineState:
294         initial_state: PipelineState = {
295             "shap_column_name": shap_column_name,
296             "erp_attribution_type": erp_attribution_type,
297             "isa_paragraph_shap": "",
298             "isa_paragraph_erp": "",
299             "enriched_explanation": None,
300             "final_output": None
301         }
302         final_state = self._graph.invoke(initial_state)
303         self.ui_dashboard.render_findings(final_state)
304         return final_state
305
```

Figure 20 LangGraph Code

Sub-Pipeline: Step 19 till Step22

With the execution environment established, the pipeline is executed in accordance with the sequence diagram (Figure 16) starting from step 19. Steps 19 to 22 cover the ISA retrieval stage, in which queries are constructed from each MergedExplanation, a similarity search is performed against the pre-indexed ISA corpus, and the results are packaged into EnrichedExplanation objects.

Step 19: The retrieval process begins as shown in Figure 22 line 157 `retrieve(merged: list[MergedExplanation])` by receiving a list of merged explanations, where each entry contains a SHAP feature name and an ERP control attribution (for the demonstration this list contains only 1 entry because we demonstrate 1 flagged row behavior) as shown in Figure 21 then calling the `retrieve_node()` as shown in Figure 22 line 165 to do similarity search for each entry.



The screenshot displays the 'ISA Audit Intelligence' web application interface. At the top, there is a header with the application name and navigation links: 'ERP Attribution', 'SHAP Analysis', 'ISA Retrieval', and 'AI Narrative'. Below the header, the 'SELECT AUDIT INPUTS' section contains two dropdown menus. The first dropdown, labeled 'ERP CONTROL ATTRIBUTE', has 'SoD Violation' selected. The second dropdown, labeled 'SHAP FEATURE COLUMN', has 'posting_hour' selected. Below these dropdowns is an orange button labeled 'Analyse & Generate Narrative'. To the right of the 'SHAP FEATURE COLUMN' dropdown, a list of available features is shown: 'posting_hour', 'amount_ratio', and 'account_deviation'.

Figure 21 UI Inputs

As illustrated in Figure 22, the ISARAGRetrieval class implements the ISA standard retrieval stage of the pipeline, exposing two methods that together realize sequence steps 20 through 22 of the conceptual model.

Step 20: The retrieve_node method on lines 169–186 accepts the shared PipelineState and constructs two independent queries. Lines 170–171 apply a split on the hyphen character to extract the terminal segment of each input field, normalizing compound names into concise query terms. Lines 172–173 then issue two separate similarity_search calls against the ISAVectorIndex, one from shap_query and one from erp_query, each returning the single (Top-1) most semantically relevant ISA paragraph for its respective input dimension.

Step 21: The retrieved paragraphs are assigned to isa_paragraph_shap on line 172 and isa_paragraph_erp on line 173, providing one ISA grounding per input dimension. Each result is a raw string returned directly from similarity_search, representing the top-ranked paragraph from the persistent ChromaDB collection as configured in the ISAVectorIndex.

Step 22: Lines 174–180 package all retrieval outputs into a ControlAttribution instance carrying the erp_attribution_type and an EnrichedExplanation instance carrying both retrieved ISA paragraphs, the original shap_column_name, and the constructed erp_attribution.

```

152 class ISARAGRetrieval:
153     def __init__(self, isa_index: ISAVectorIndex):
154         self.isa_index = isa_index
155
156     def retrieve(
157         self, merged: list[MergedExplanation]
158     ) -> list[EnrichedExplanation]:
159         results = []
160         for entry in merged:
161             state = {
162                 "shap_column_name": entry.shap_column_name,
163                 "erp_attribution_type": entry.erp_attribution.attribution_type
164             }
165             node_result = self.retrieve_node(state)
166             results.append(node_result["enriched_explanation"])
167         return results
168
169     def retrieve_node(self, state: PipelineState) -> dict:
170         shap_query = state["shap_column_name"].split('-')[-1] if '-' in state["shap_column_name"] else state["shap_column_name"]
171         erp_query = state["erp_attribution_type"].split('-')[-1] if '-' in state["erp_attribution_type"] else state["erp_attribution_type"]
172         isa_paragraph_shap = self.isa_index.similarity_search(shap_query)
173         isa_paragraph_erp = self.isa_index.similarity_search(erp_query)
174         erp_attribution = ControlAttribution(
175             attribution_type=state["erp_attribution_type"]
176         )
177         enriched = EnrichedExplanation(
178             isa_paragraph=isa_paragraph_shap,
179             shap_column_name=state["shap_column_name"],
180             erp_attribution=erp_attribution
181         )
182         return {
183             "isa_paragraph_shap": isa_paragraph_shap,
184             "isa_paragraph_erp": isa_paragraph_erp,
185             "enriched_explanation": enriched
186         }
187

```

Figure 22 Retrieval Code

Sub-Pipeline: Step 23 till Step 24

As illustrated in Figure 23, the LocalLLM class implements the narrative generation stage, with class-level attributes on lines 192–194 fixing the model identifier to llama3.1:8b-instruct-q4_K_M, capping ISA input length at 4000 characters via max_chars, and bounding generation time via timeout at 120 seconds.

Step 23: The generate_node method on lines 236–246 serves as the LangGraph-bound entry point. It extracts the enriched_explanation from the shared PipelineState on line 240 and delegates to generate on lines 241–244, passing the enriched explanation as a single-element list alongside the isa_paragraph_erp field from state. The resulting output is assigned on line 245 and returned as a PipelineState update carrying final_output on line 246.

Step 24: The generate method on lines 196–234 iterates over each EnrichedExplanation entry and constructs a structured prompt on lines 203–222 that explicitly constrains the model to a formatting role, injecting shap_column_name on line 214, erp_attribution_type on line 216, the SHAP-matched ISA paragraph truncated to max_chars on line 218, and the ERP-matched ISA paragraph similarly truncated on line 220. The prompt is submitted via HTTP POST to the locally hosted Ollama endpoint on lines 223–231, with stream set to False and timeout passed directly to bound generation time. Line 232 raises on non-2xx responses, and line 233 extracts the narrative string from the JSON reply and appends it to outputs, which is returned on line 234 as list[str].

```

191 class LocalLLM:
192     llm_model: str = "llama3.1:8b-instruct-q4_K_M"
193     max_chars: int = 4000
194     timeout: int = 120
195
196     def generate(
197         self,
198         enriched: list[EnrichedExplanation],
199         isa_paragraph_erp: str = ""
200     ) -> list[str]:
201         outputs = []
202         for entry in enriched:
203             prompt = (
204                 "You are an experienced financial auditor "
205                 "writing a formal audit observation. "
206                 "Use only the information provided below. "
207                 "Do not add facts, figures, or conclusions "
208                 "that are not explicitly stated in the context. "
209                 "Do not speculate or introduce information "
210                 "from outside the provided context. "
211                 "Output only the audit observation text "
212                 "with no preamble or additional commentary.\n\n"
213                 f"SHAP Feature: "
214                 f"{entry.shap_column_name}\n"
215                 "ERP Control: "
216                 f"{entry.erp_attribution.attribution_type}\n"
217                 "ISA Basis (SHAP): "
218                 f"{entry.isa_paragraph[:self.max_chars]}\n"
219                 "ISA Basis (ERP): "
220                 f"{isa_paragraph_erp[:self.max_chars]}\n\n"
221                 "Audit Observation:"
222             )
223             response = requests.post(
224                 "http://localhost:11434/api/generate",
225                 json={
226                     "model": self.llm_model,
227                     "prompt": prompt,
228                     "stream": False
229                 },
230                 timeout=self.timeout
231             )
232             response.raise_for_status()
233             outputs.append(response.json().get("response", ""))
234         return outputs
235
236     def generate_node(
237         self,
238         state: PipelineState
239     ) -> dict:
240         enriched = state["enriched_explanation"]
241         outputs = self.generate(
242             [enriched],
243             isa_paragraph_erp=state["isa_paragraph_erp"]
244         )
245         final_output = outputs[0] if outputs else ""
246         return {"final_output": final_output}
247

```

Figure 23 LLM Generation Prompt Code

Sub-Pipeline: Step 25

Receiving the full state allows the demonstration UI to display the two ISA retrieval results first followed by the LLM narrative as shown in Figure 24 and 25.

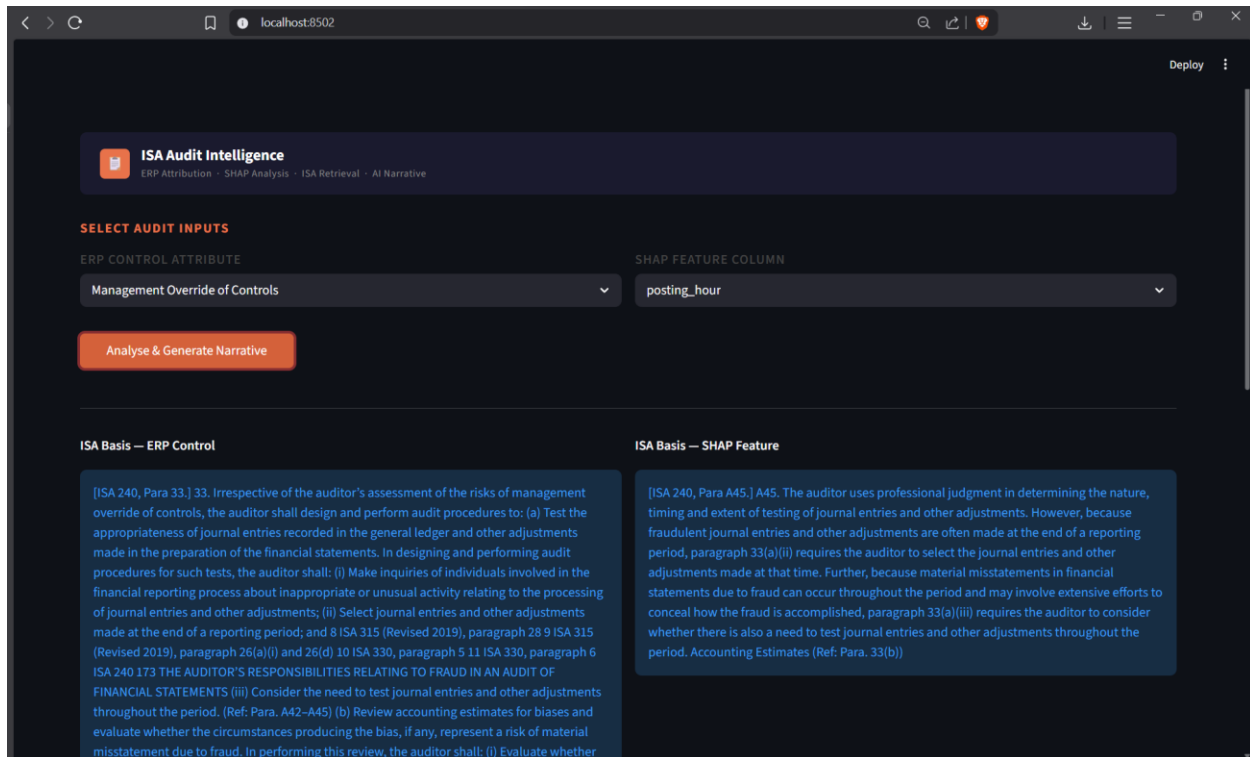


Figure 24 Demonstration UI-1

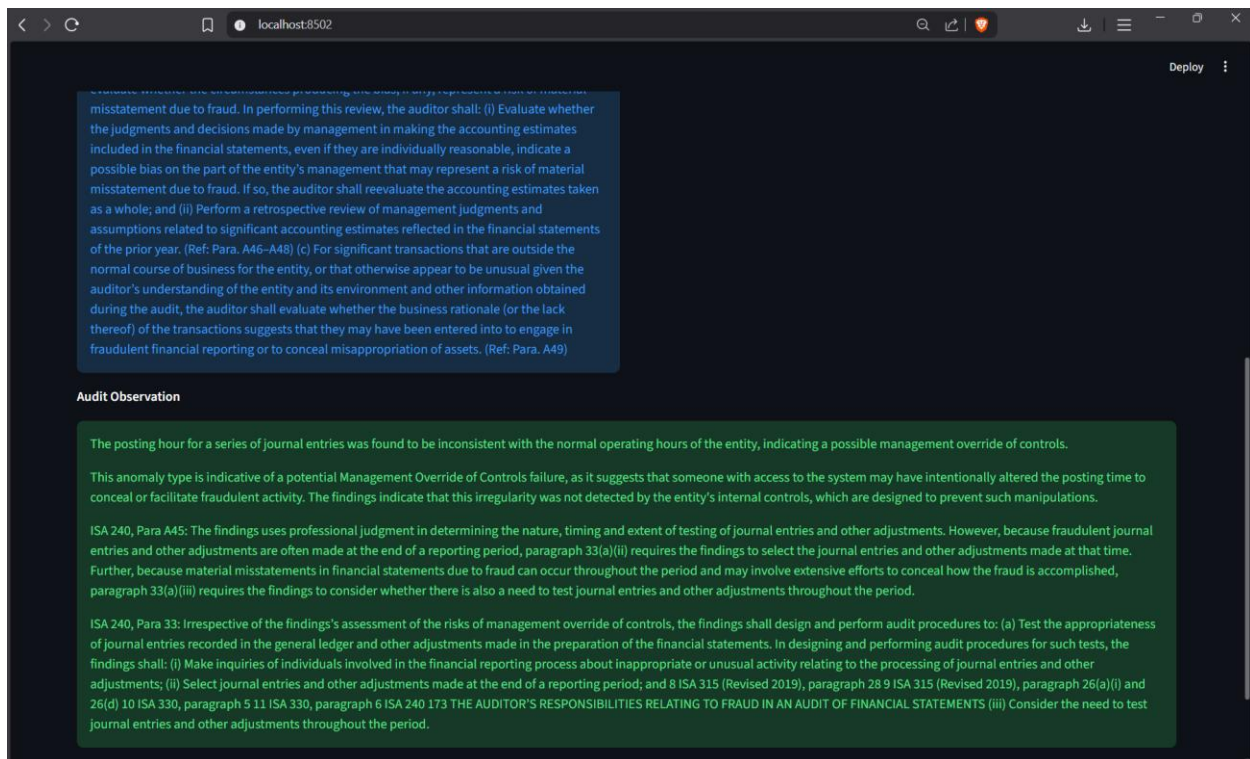


Figure 25 Demonstration UI-2

7.3.5 Verification and Validation

This section evaluates the prototype through two distinct assessments. Verification checks whether the implemented prototype conforms to the design specification defined by the class diagram and sequence diagram in Section 7.2.2, confirming that the system was built right. Validation checks whether the prototype fulfils the functional and non-functional requirements defined in Sections 7.2.2.1 and 7.2.2.2, confirming that the right system was built. Both assessments are scoped to the implemented demonstration covering steps 19 to 25, as declared in Section 7.3.1. Requirements FR1 to FR7, FR10, NFR2, NFR3 and NFR6 are excluded because they are not implemented in the prototype. FR8 is addressed separately through the retrieval confusion matrix analysis.

Verification

As shown in Table 5, each test targets a component within the demonstration scope and assesses whether its behavior matches the design specification, confirming that the system was built right. Expected behavior is derived from the class diagram and sequence diagram. Observed behavior is drawn from the prototype source code and the UI dashboard.

Test ID	Component	Expected behavior (spec)	Observed behavior (prototype)	Result
Ver-1	LangGraph Orchestrator	Constructs a StateGraph over PipelineState; registers retrieve and generate nodes; compiles to executable graph; exposes start_pipeline () as entry point	build_graph () creates StateGraph (PipelineState), adds retrieve_isa and generate_narrative nodes with directed edge, compiles graph. start_pipeline () constructs initial state and invokes compiled graph	Pass
Ver-2	Corpus chunker	ISA PDFs are split into paragraph-level chunks using numbered identifiers (e.g., 33., A42.) as boundaries	Line-by-line scan via pdfplumber detects numbered paragraph starts; all subsequent lines until next identifier are merged into one chunk	Pass
Ver-3	LocalLLM	Model acts as text renderer only; output capped by max_chars and timeout; returns final_outputs: list[str]	Prompt explicitly instructs model to format pre-determined facts without independent reasoning; max_chars truncates ISA inputs; timeout parameter is passed to Ollama request; response parsed from JSON and appended to output list	Pass
Ver-4	ISAVectorIndex	Stores embeddings using embedding_model, similarity_metric, and index_name; returns the nearest paragraph per query	ChromaDB collection is configured with BAAI/bge-large-en-v1.5, cosine distance, and a named collection. Persistent Client persists across restarts. Two independent queries return two paragraph strings	Pass

Test ID	Component	Expected behavior (spec)	Observed behavior (prototype)	Result
Ver-5	ISARAG Retrieval	Receives a list [MergedExplanation]; constructs two queries per entry one from shap_column_name and one from erp_attribution.attribution_type; calls similarity search on ISAVectorIndex	UI passes auditor-selected ERP type and SHAP feature directly into retrieve_node(); two similarity_search() calls are issued against the ChromaDB collection, faithfully simulating the dual-query design	Pass

Table 5 Verification Tests

Validation

As shown in Table 6, each row maps one requirement from Section 7.2.2 to evidence produced by the demonstration, confirming that the right system was built. Each requirement is assessed within the declared scope and simplifications of Section 7.3.1, and a confusion matrix for the retrieval process is constructed to evaluate FR8.

Req. ID	Evidence	Result
FR-9	LocalLLM.generate_node () sends structured prompt to Ollama (llama3.1:8b-instruct-q4_K_M); prompt explicitly constrains model to formatting role; narrative string returned and rendered on dashboard	Pass
NFR-1	Ollama serves llama3.1:8b-instruct-q4_K_M locally via localhost HTTP; ChromaDB persists to local disk; sentence-transformer runs locally via PyTorch; no external API calls are made	Pass
NFR-4	LangGraph StateGraph decouples each node; ISARAGRetrieval, LocalLLM, and UI_Dashboard are injected as dependencies into LangGraphOrchestrator; swapping any one does not require modifying others	Pass
NFR-5	Prompt explicitly instructs model to act as text renderer only; ISA inputs truncated to max_chars; timeout passed to Ollama HTTP request; no independent reasoning is prompted	Pass
NFR-7	ChromaDB collection is configured with fixed embedding_model (BAAI/bge-large-en-v1.5), similarity_metric (cosine), and index_name; index is persisted to disk and reused across restarts. Formal version control of config is not implemented in the prototype	Pass

Table 6 Validation Tests

Retrieval Evaluation via Confusion Matrix Analysis (FR-8)

Since no external labelled benchmark exists for ISA-grounded audit queries, a ground truth set was constructed through expert annotation. Each SHAP value and ERP attribute was manually assigned to its correct ISA paragraph based on semantic alignment between the query and the paragraph regulatory content.

SHAP Feature Retrieval

As shown in Figure 26, this evaluation uses a focused sub-metric that captures only true positive retrievals per SHAP feature name rather than all false positives and false negatives across every ISA paragraph combination, a deliberate choice to simplify scope and concentrate on whether each feature name consistently maps to its correct paragraph instead of exhaustively measuring misrouting across all paragraph permutations. Each ISA X Para Y label represents the correct retrieval target for one of the three SHAP feature names available in the dropdown. ISA 330 Para 18 and ISA 240 Para A45 each achieved 3/3 correct retrievals, while ISA 240 Para 23 achieved only 1/3, with the two remaining queries misrouted to ISA 315 Para A27 due to semantic overlap in the vector space. Using the accuracy formula from Section 4.2.1, Equation 1, **overall retrieval accuracy = 7 ÷ 9 = 77.78%.**

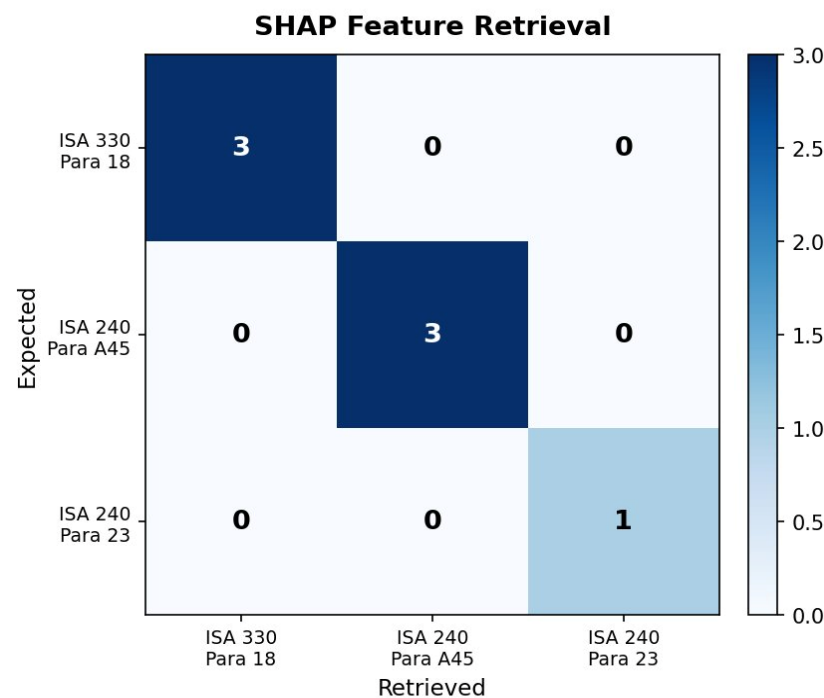


Figure 26 SHAP Feature Retrieval Confusion Matrix

ERP Attribution Retrieval

As shown in Figure 27, ISA 265 Para 8, ISA 330 Para 8, and ISA 240 Para 33 each achieved 2 out of 3 correct retrievals across all their associated inputs, with the remaining queries not captured within this sub-matrix. As per the accuracy formula defined in Section 4.2.1, Equation 1, **the overall retrieval accuracy is computed as $6 \div 9 = 66.67\%$.**

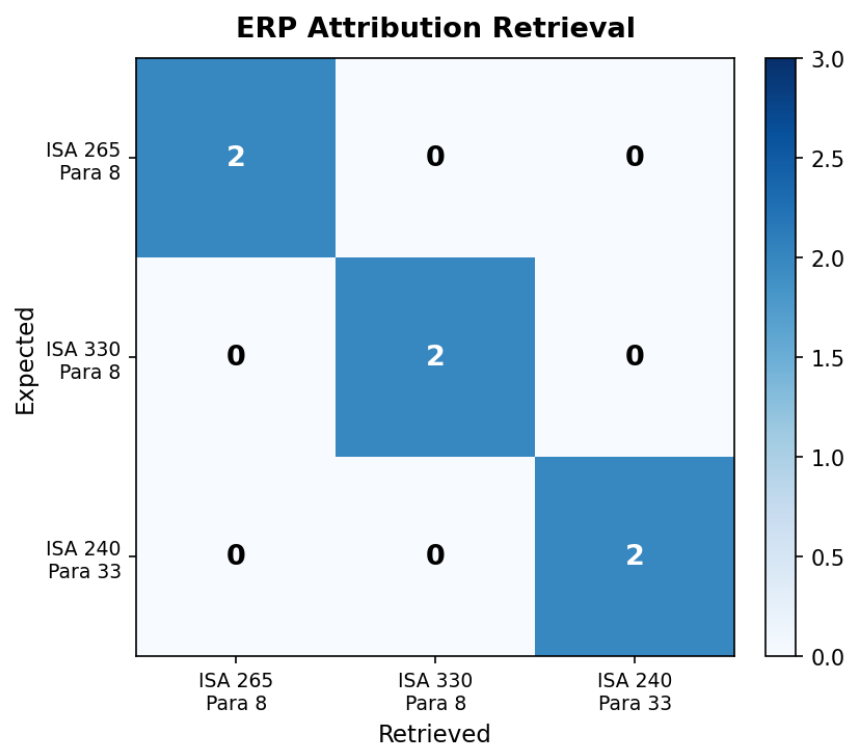


Figure 27 ERP Attribution Retrieval Confusion Matrix

The narrative output of the local language model is intentionally excluded from this evaluation. As established in the system design, the model operates strictly as a text renderer formatting pre-determined inputs rather than performing independent inference, meaning its output quality is wholly contingent on the correctness of the retrieval stage. Given that retrieval stage correctly identified the relevant ISA paragraph in 77.78% of SHAP queries and 66.67% of ERP attribution queries. This means that most generated narratives are factually grounded, though some mismatches remain and are acknowledged as a limitation in Section 8.2.2.

7.3.6 Expert Interview

In this section an expert interview is conducted to evaluate the conceptual model and the demonstration prototype from multiple professional perspectives, structured around the TOE (Technological, Organizational, and Environmental) framework (Baker, 2011), and administered using the Delphi method. Four domain experts who do not know each other are consulted: an AI Engineer from a telecommunications company, a Full-Stack Software Engineer from an FMCG company, an External Auditor from an accounting firm, and an Internal Auditor from an accounting firm. Each expert is presented with the sequence diagram, the class diagram, and the demonstration output. A total of eight structured questions were administered across two distinct expert groups. The first group comprised a full-stack engineer and an AI engineer, who were asked four technical questions addressing model architecture and implementation, representing the technological dimension of the TOE framework. The second group consisted of an internal auditor and an external auditor, who were asked four domain-specific questions pertaining to audit methodology, representing the organizational and environmental dimensions of the TOE framework.

Expert 1: AI Engineer at a Telecommunications Company

Q1: Does the overall pipeline architecture reflect sound software design principles in terms of component separation and responsibility assignment?

Yes. Each component carries a clearly scoped responsibility and the dependency injection pattern used across the orchestrator ensures loose coupling throughout the pipeline.

Q2: Is the use of LangGraph as an orchestration framework appropriate for this type of sequential AI pipeline?

Yes. The StateGraph abstraction makes execution order explicit and auditable, and the shared PipelineState provides a clean mechanism for passing structured data between nodes.

Q3: Does the ISA RAG retrieval design reflect a valid and practical retrieval approach?

Yes. Constructing separate queries from the SHAP feature name and ERP control attribution type ensures each dimension is independently grounded in the most semantically relevant ISA paragraph.

Q4: Does the sequence diagram accurately represent the interaction flow between system components?

The sequence diagram has no error handling for Step 23-24. If the locally hosted Ollama model is unavailable or times out, the pipeline silently fails with no feedback to the auditor. An ALT combined fragment should be added covering Steps 23-24 showing two branches: the success path (LLM renders narrative, returns final_outputs: list[str]) and the failure path (LLM call fails, UI_Dashboard displays an error message to the auditor).

Expert 2: Full-Stack Software Engineer at a FMCG Company

Q1: Does the overall pipeline architecture reflect sound software design principles in terms of component separation and responsibility assignment?

Yes. Component boundaries are well defined and the decision to keep UI_Dashboard.render_findings() outside the LangGraph execution graph is a sound choice that preserves correct output ordering.

Q2: Is the use of LangGraph as an orchestration framework appropriate for this type of sequential AI pipeline?

Yes. The compiled StateGraph provides an explicit and inspectable execution structure.

Q3: Does the ISA RAG retrieval design reflect a valid and practical retrieval approach?

Yes. Using ChromaDB with cosine similarity and BAAI/bge-large-en-v1.5 is appropriate for semantic search over structured regulatory text, and the persistent index avoids repeated embedding overhead across sessions.

Q4: Does the sequence diagram accurately represent the interaction flow between system components?

Yes. The component interactions, message types, and execution order are consistent with both the class diagram and the implemented prototype. No changes are recommended.

Expert 3: External Auditor at an Accounting Firm

Q1: Does the combined output of SHAP attribution, ERP control identification, and ISA citation provide sufficient transparency for professional audit judgment?

Yes. The three elements together provide a traceable basis for audit judgment. SHAP identifies the contributing features, ERP attribution links the finding to a specific control failure, and the ISA citation grounds it in a recognized professional standard.

Q2: Are the five selected ISA standards appropriate and sufficient for General Ledger audit coverage?

Yes. The selected standards collectively address fraud risk, internal control deficiencies, risk assessment, auditor responses, and sampling, which covers the primary dimensions relevant to GL anomaly detection. No additions are necessary.

Q3: Does the pipeline align with standard audit procedures and professional auditing practice?

Yes. The pipeline aligns with the planning and risk assessment phases of standard audit engagement and positions the system as a tool that informs auditor judgment rather than replacing it.

Q4: Would the generated narrative be practically usable as supporting audit documentation?

Yes. Because the language model acts solely as a text renderer formatting pre-determined input, the output constitutes a structured finding rather than an autonomous opinion, which makes it professionally acceptable as audit working paper documentation.

Expert 4: Internal Auditor at an Accounting Firm

Q1: Does the combined output of SHAP attribution, ERP control identification, and ISA citation provide sufficient transparency for professional audit judgment?

Yes. The output directly satisfies the internal audit requirement for clear, evidence-based, and traceable findings and would support the preparation of audit reports without significant manual reformatting.

Q2: Are the five selected ISA standards appropriate and sufficient for General Ledger audit coverage?

Yes. ISA 240 and ISA 315 are the most relevant for fraud risk and control environment assessment, which are the primary concerns in GL anomaly detection. The selection is appropriate for the declared scope.

Q3: Does the pipeline align with standard audit procedures and professional auditing practice?

Yes. The flow from data ingestion through anomaly detection to narrative generation mirrors the fieldwork and reporting phases of a standard internal audit engagement and is consistent with IIA expectations for transparent and defensible findings.

Q4: Would the generated narrative be practically usable as supporting audit documentation?

Yes. Every output string is derived from pre-determined SHAP values, a specific ERP control failure, and a cited ISA paragraph, making it directly suitable for inclusion in audit working papers.

7.3.7 Refinement

Following the expert interview, one modification to the conceptual model was identified based on the feedback provided by the AI Engineer. Although this concern was raised by one expert only, it is considered significant given the operational risk of silent pipeline failures in locally hosted inference environments. The sequence diagram and demonstration are therefore updated accordingly to address the absence of error handling in the LLM generation stage.

7.3.7.1 Conceptual Model Refinement

The sequence diagram was revised as shown in Figure 28 to incorporate an ALT combined fragment encompassing Steps 23 and 24. The upper branch, guarded by the condition [LLM available], retains the original interaction in which the LangGraphOrchestrator invokes `generate(enriched: list[EnrichedExplanation])` on the LocalLLM component and receives `final_outputs: list[str]` in return. The lower branch, guarded by the condition [LLM unavailable], captures the failure path in which the LLM call does not complete successfully and an `error: str` is returned to the LangGraphOrchestrator, which halts the pipeline without proceeding to the rendering step. This modification ensures that the conceptual model explicitly accounts for LLM unavailability, which represents a realistic operational risk in locally hosted inference environments.

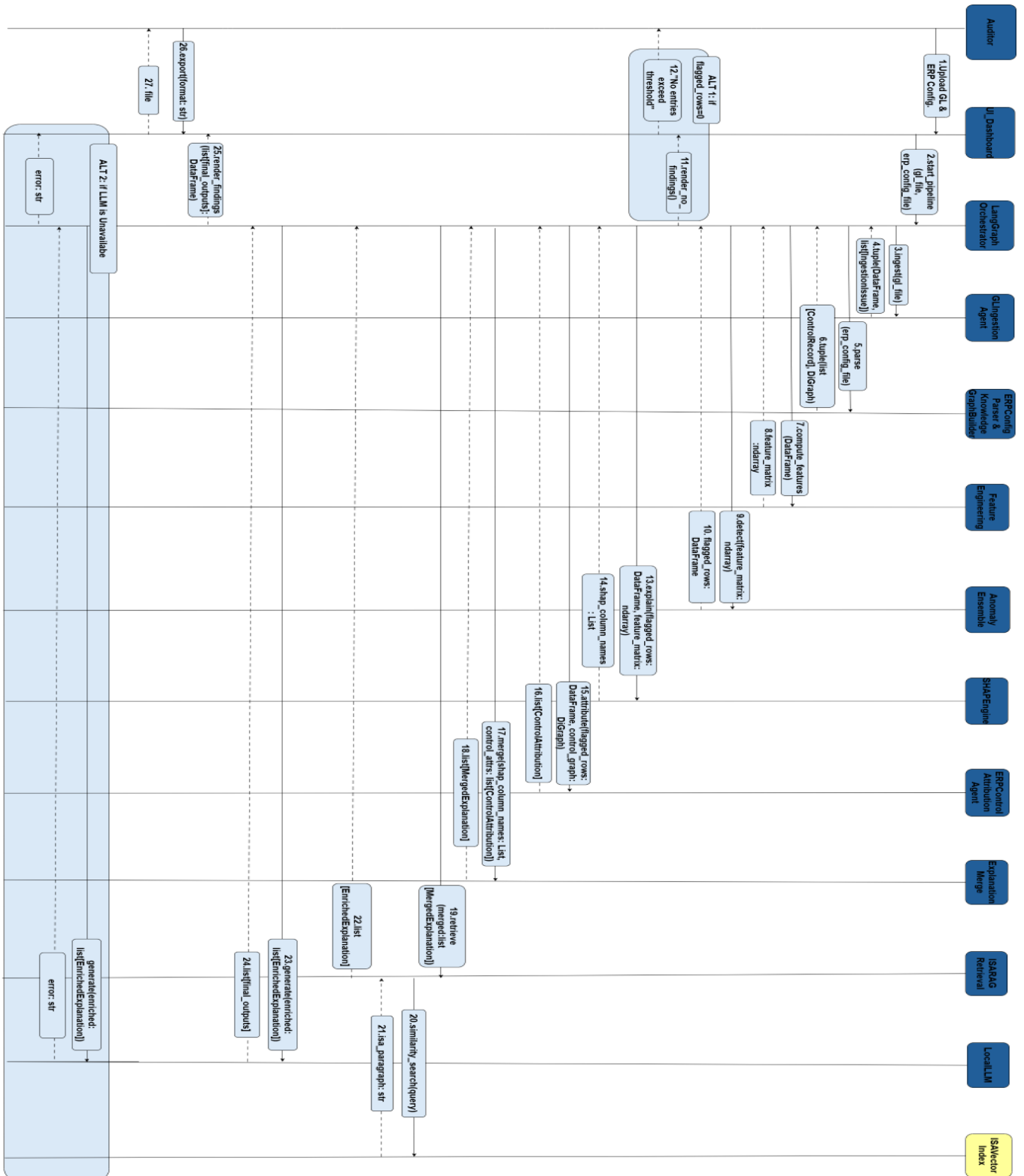


Figure 28 Sequence Diagram Refinement

7.3.7.2 Demonstration Refinement

As illustrated in Figure 29, the HTTP POST call is wrapped within a try/except block handling three distinct failure cases. A `ConnectionError` on lines 244–247 appends a message indicating the Ollama service is unreachable, a `Timeout` on lines 249–252 indicates the model exceeded the generation window, and a general `Exception` fallback on lines 254–255 captures all remaining failures, collectively preventing silent pipeline crashes. The `generate_node` method on lines 258–271 is correspondingly updated to inspect the output on line 268 and, upon detecting an `ERROR:` prefix, invoke `st.error()` on line 269 to surface the failure on the auditor-facing dashboard before returning an empty `final_output` on line 270.

```
231         try:
232             response = requests.post(
233                 "http://localhost:11434/api/generate",
234                 json={
235                     "model": self.llm_model,
236                     "prompt": prompt,
237                     "stream": False,
238                     "options": {"temperature": 0}
239                 },
240                 timeout=self.timeout
241             )
242             response.raise_for_status()
243             outputs.append(response.json().get("response", ""))
244         except requests.exceptions.ConnectionError:
245             outputs.append(
246                 "ERROR: Local LLM is unavailable. "
247                 "Please ensure Ollama is running and try again."
248             )
249         except requests.exceptions.Timeout:
250             outputs.append(
251                 "ERROR: LLM request timed out. "
252                 "The model took too long to respond."
253             )
254         except Exception as e:
255             outputs.append(f"ERROR: Narrative generation failed. {str(e)}")
256         return outputs
257
258     def generate_node(
259         self,
260         state: PipelineState
261     ) -> dict:
262         enriched = state["enriched_explanation"]
263         outputs = self.generate(
264             [enriched],
265             isa_paragraph_erp=state["isa_paragraph_erp"]
266         )
267         final_output = outputs[0] if outputs else ""
268         if final_output.startswith("ERROR:"):
269             st.error(final_output)
270             return {"final_output": ""}
271         return {"final_output": final_output}
272
```

Figure 29 Demonstration Refinement Code

As demonstrated in Figure 30, Ollama is deliberately turned off locally to test the refinement, upon which the dashboard surfaces the connection failure inline, displaying the message "Local LLM is unavailable. Please ensure Ollama is running and try again" beneath the input controls. The ISA Basis panels remain populated, while the Audit Observation section returns an empty narrative, confirming that the error handling mechanism prevents a silent pipeline crash.

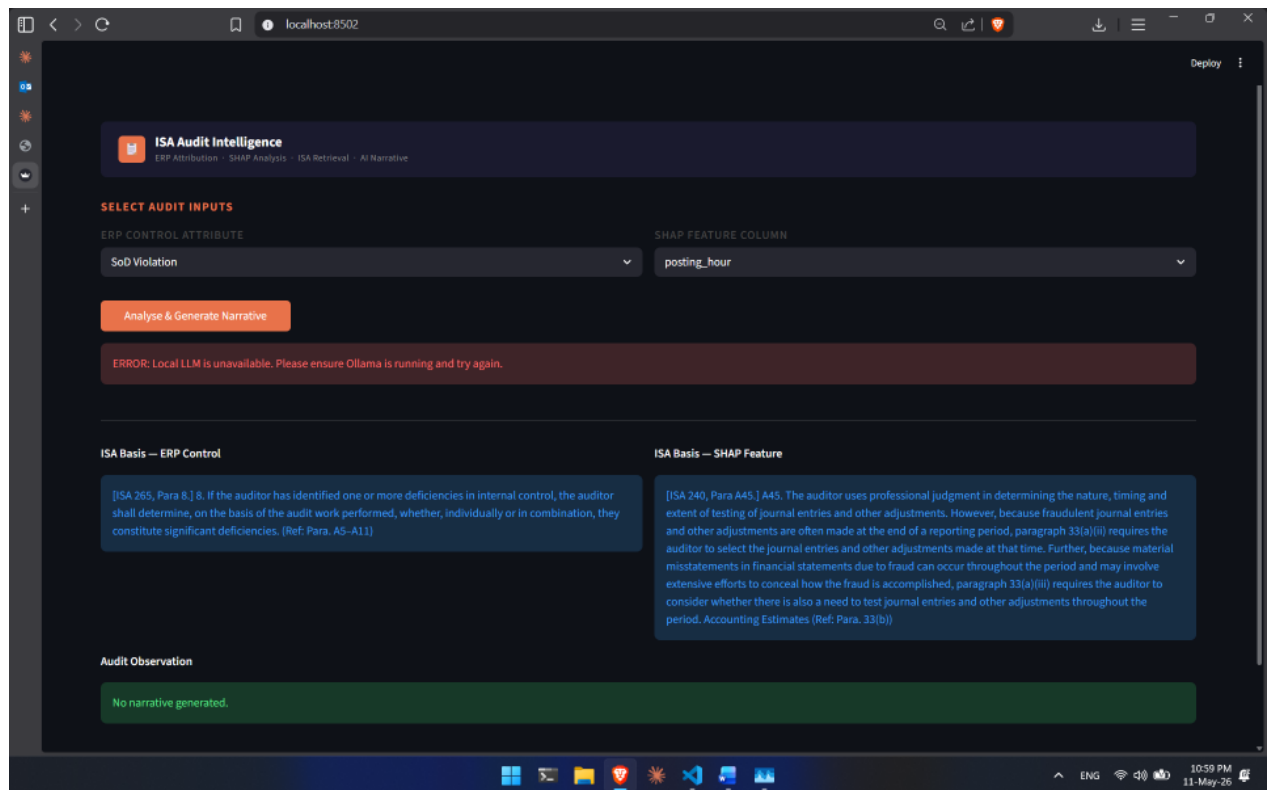


Figure 30 UI after Refinement

Following the refinement of the conceptual model and the corresponding update to the demonstration, the modified sequence diagram and updated code were presented to the two technical experts only, as the refinement is technical in nature. In accordance with the iterative nature of the Delphi method, feedback was collected from both the AI Engineer and the Full-Stack Software Engineer, both of whom confirmed that the revised conceptual model and demonstration adequately addressed the concern raised during the initial interview.

8 Conclusion

This chapter presents the concluding remarks of the study. First, a summary of the work conducted across all chapters is provided. This is followed by a discussion of the limitations encountered, directions for future work, and a comparison with related studies.

8.1 Summary

The main objective of this research is to **develop a conceptual model** for integrating Explainable Artificial Intelligence (XAI) into the financial auditing process, addressing the identified scientific gap, through which audit transparency, interpretability, and reliability can be enhanced to improve the overall quality of AI-supported audit processes.

It is **confirmed** that this objective has been achieved. The following presents the work conducted across each chapter as evidence of this achievement.

Chapter 1 presented the introduction of the study, covering the motivation behind integrating AI into financial auditing, the problem definition centered on the black-box nature of existing AI systems, the identified scientific gap reflecting the absence of a transparent and standards-aligned AI artifact for auditing, the formulated research question, the research objective of developing an XAI-based conceptual model, the methods adopted including Design Science Research and expert-based evaluation, and the overall structure of the paper across all eight chapters.

Chapter 2 established the theoretical foundation of financial auditing by defining auditing as the systematic and independent examination of financial records to determine whether they present a true and fair view of an organization's financial position. The core types of auditing were presented, namely internal, external, and forensic auditing, along with the structured audit process that spans planning, implementation, monitoring, and review. Audit quality and the limitations of traditional audit methods were also examined, demonstrating that conventional sampling-based approaches are insufficient to handle the scale and complexity of modern financial data.

Chapter 3 introduced the main concepts of artificial intelligence that underpin the proposed system. Machine learning, deep learning, natural language processing, explainable artificial intelligence, and generative AI were presented, with particular attention given to XAI and its role in making AI decisions transparent and understandable. Large Language Models were introduced as the generative component responsible for producing human-readable audit narratives. The advantages and disadvantages of AI were also reviewed, with the black-box problem identified as the core challenge motivating this study.

Chapter 4 examined how different AI techniques are applied specifically in auditing activities. Large Language Models were discussed in the context of preparing audit working papers and supporting audit planning. Supervised techniques, including Support Vector Machines and Decision Trees, were presented for their use in fraud detection and financial classification. Unsupervised techniques, including K-Means clustering and unsupervised NLP anomaly detection, were introduced for identifying unusual transactions without labeled data. Autoencoders were discussed as a deep learning method for detecting anomalies in high-dimensional financial data.

Chapter 5 presented a structured literature review through which eleven candidate papers were identified and filtered to five peer-reviewed publications from 2021 to 2025. The state of the art was summarized, and a comparative discussion was conducted across four dimensions: goals, gaps, methods used, and limitations. The scientific gap was then identified: existing AI systems in auditing operate as black boxes, producing outputs that auditors cannot easily understand, justify, or align with professional standards. This gap led to the formulation of the research question and the research objective of this study.

Chapter 6 presented the scientific methodology adopted for this study. Several research methodologies were introduced and discussed, including Design Science Research, data science process models such as CRISP-DM and KDD, qualitative and quantitative research approaches, the Delphi method, and literature review methods. Based on the research objective of developing a conceptual model, Design Science Research was selected as the most suitable primary research paradigm, following the structured process model proposed by Offermann et al. (2009), which organizes the research into three phases: problem identification, design, and evaluation. The chapter concluded with the solution approach diagram that guided the full implementation.

Chapter 7 presented the full design, implementation, and evaluation of the proposed system. A twenty-seven-step end-to-end conceptual pipeline was designed and described from both a structural perspective using a UML class diagram and a behavioral perspective using a sequence diagram. A focused prototype covering steps 19 to 25 was implemented using Python, LangGraph, ChromaDB, and a locally hosted Llama 3.1 8B model. The prototype was evaluated through verification and validation tests, a retrieval confusion matrix analysis, and expert interviews with four domain professionals following the Delphi method. All tests passed, one refinement was introduced based on expert feedback, and the final model was confirmed as aligned with standard audit procedures and suitable for professional audit documentation.

8.2 Limitations

Despite the contributions described above, this study is subject to a number of limitations that must be acknowledged. These limitations are categorized into two types: Environmental limitations which are practical constraints imposed by the research environment, such as time and hardware availability while Conceptual limitations are inherent boundaries in the proposed solution itself, regardless of the environment.

8.2.1 Environmental Limitations

The scope and depth of the prototype were constrained by the limited time and computational resources available during the development of this thesis. Access to dedicated hardware was restricted to a relatively low powered GPU with only 6 GB VRAM, requiring the use of a 4-bit quantized language model variant (llama3.1:8b-instruct-q4_K_M) to remain within the available memory capacity. Similarly, the embedding model was selected based on hardware feasibility rather than optimal retrieval performance, as more advanced embedding models exceeded the available computational and memory resources. In addition, ERP system configuration data and real general ledger (GL) data were excluded from the demonstration prototype because both require privileged enterprise access.

8.2.2 Conceptual Limitations

A fundamental conceptual limitation concerns the role of artificial intelligence in the audit process. Although the proposed system is designed to improve transparency and explainability, it cannot and should not replace the professional judgment of a qualified auditor. AI and machine learning techniques, even when augmented with explainability mechanisms such as SHAP and RAG-based standard retrieval, remain tools that support rather than substitute human oversight. Auditing is a professional, regulatory, and legal activity in which decisions must ultimately be made and defended by a certified human practitioner. Any AI system applied in this domain must therefore be treated as a decision-support instrument, and its output must be reviewed, interpreted, and approved by the responsible auditor before they are used in formal audit documentation or professional opinions.

A further conceptual limitation relates to the integrity of the ground truth labels used in retrieval evaluation. Since no external labelled benchmark exists for ISA-grounded audit queries, ground truth was constructed through expert annotation, where each query was manually assigned to its correct ISA paragraph based on semantic alignment. This introduces inherent subjectivity and the possibility of annotation inconsistencies, meaning retrieval metrics cannot be considered absolutely precise and should be interpreted with this caveat in mind.

8.3 Future Work

Several directions are identified for extending and improving this work in the future. First, the adoption of more capable language models is recommended. The current prototype relies on a quantized 8-billion-parameter model to satisfy the hardware constraints described above. Future implementations could benefit from deploying larger, higher-precision models that are capable of producing richer, more contextually accurate audit narratives, particularly when the number of flagged entries and the complexity of the ERP configurations increase.

Second, the anomaly detection ensemble should be extended from unsupervised to supervised training on organization-specific corporate data. The current ensemble operates in an unsupervised manner on general financial data because labeled fraud datasets representing specific corporate environments were not available during this study. Once a sufficient volume of labeled GL entries is collected from a target organization, the detection models could be fine-tuned on that data, substantially improving the precision and recall of the anomaly detection stage for the specific transactional patterns and control structures of that organization.

Third, deeper integration with live ERP systems is proposed. In the current conceptual model, ERP control attributes are supplied through a static configuration export. Future work could establish a direct connection to the ERP system's API or data layer, enabling the pipeline to access a richer and continuously updated set of control attributes, authorization rules, and organizational hierarchy data. This would allow the SHAP attributions and ISA citations to be grounded in a far more comprehensive knowledge of the organization's internal control environment, producing audit findings that are more specific, actionable, and aligned with actual operational risk.

8.4 Related Work

The proposed work is compared in this section against three closely related studies that represent the current state of the art in AI-assisted financial and transaction auditing.

Bai et al. (2025) present AuditAgent, a multi-agent LLM framework designed to detect fraudulent evidence across multiple financial documents by combining expert-guided reasoning with cross-document retrieval. Na et al. (2025) proposed HCLA, a human-centered LLM-agent system for detecting anomalous digital asset transactions, emphasizing human-in-the-loop verification as a key design principle. Watson (2025) introduces a graph-based anomaly detection approach for cryptocurrency transactions that uses LLMs to generate natural language explanations of graph-derived anomaly scores, advocating an "explain first, trust later" philosophy.

While each of these studies makes a meaningful contribution, the system proposed in this thesis offers two distinct advantages over all three.

First, the proposed system is the only one that grounds its AI-generated explanations in formal auditing standards. AuditAgent, HCLA, and Watson all produce findings or explanations that are expressed in general natural language without any reference to an authoritative regulatory or professional standard. In the proposed pipeline, every narrative output is directly linked to a specific ISA paragraph retrieved through semantic search, ensuring that each finding is not only explainable but also professionally defensible and aligned with internationally recognized audit requirements. This characteristic is essential for any system intended for use in a real audit engagement.

Second, the proposed system is the only one designed for integration with enterprise ERP environments. The three related studies focus on document-level fraud detection or cryptocurrency transaction analysis, neither of which accounts for the organizational control structures that are central to corporate financial auditing. The proposed pipeline incorporates ERP configuration data as an explicit input, enabling the system to attribute anomalies to specific internal control failures within the organization's ERP environment. This enterprise context-awareness makes the proposed system significantly more applicable to the structured corporate audit settings in which professional auditors operate.

References

- Alan R. Hevner, Salvatore T. March, Jinsoo Park, Sudha Ram; Design Science in Information Systems Research¹. MIS Quarterly 1 March 2004; 28 (1): 75–10
- APAK, R., & Ganji, F. (2025). Using decision tree algorithms and artificial intelligence to increase audit quality: A data-based approach to predicting financial risks. *International Journal of Business Management and Entrepreneurship*
- Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., ... & Herrera, F. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities, and challenges toward responsible AI. *Information fusion*, 58, 82-115.
- Bai, S., Wu, B., Zhang, Y., Wu, C., Zheng, X., Yuan, Y., Wu, K., & Li, J. (2025). AuditAgent: Expert-guided multi-agent reasoning for cross-document fraudulent evidence discovery. *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF '25)*.
- Baker, Jeff. (2011). The Technology–Organization–Environment Framework. 10.1007/978-1-4419-6108-2_12.
- Chiromo, F., & Msibi, N. N. (2023). Internal audit program planning and implementation determinants of an automotive company's ISO 9001 quality system. *Journal of Applied Research on Industrial Engineering*, 10(3), 492–505.
- DeAngelo, L. E. (1981). Auditor size and audit quality. *Journal of Accounting and Economics*, 3(3), 183–199.
- Dos, D. L. D. J. S., & Dos Santos, G. C. (2025). Technological convergence in financial auditing: A systematic literature review. *Data Science in Finance and Economics*, 5(4), 440-465.
- Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). The KDD process for extracting useful knowledge from volumes of data. *Communications of the ACM*, 39(11), 27-34.
- Fink, A. (1998). *Conducting research literature reviews: From paper to the internet*. Sage Publications.
- Fırat, Z. (2025). ARTIFICIAL INTELLIGENCE IN AUDITING: OPPORTUNITIES, CHALLENGES, AND FUTURE DIRECTIONS. *Muhasebe Bilim Dünyası Dergisi*, 27(2), 77-95.
- Fotoh, Lazarus Elad & Mugwira, Tatenda, 2025. "Exploring Large Language Models in external audits: Implications and ethical considerations," *International Journal of Accounting Information Systems*, Elsevier, vol. 56(C).

- Gillani, N., Eynon, R., Chiabaut, C., & Finkel, K. (2023). Unpacking the “Black Box” of AI in education. *Educational Technology & Society*, 26(1), 99-111.
- Håkansson, A., & Phillips-Wren, G. (2024). Generative AI and large language models: Benefits, drawbacks, future, and recommendations. *Procedia Computer Science*, 246, 5458–5468.
- Hamzah, P., Yeba, E., Maithy, S. P., & Poetra, G. B. (2024). Opportunities and Challenges in Integrating Artificial Intelligence into Financial Auditing. *Journal of Economic Education and Entrepreneurship Studies*, 5(4), 591-600.
- Jain, S., Howe, B., Yan, J., & Cruanes, T. (2018). Query2vec: An evaluation of NLP techniques for generalized workload analytics. *arXiv preprint arXiv:1801.05613*.
- Janiesch, C., Zschech, P., & Heinrich, K. (2021). Machine learning and deep learning. *Electronic markets*, 31(3), 685-695.
- Khanzode, K. C. A., & Sarode, R. D. (2020). Advantages and disadvantages of artificial intelligence and machine learning: A literature review. *International Journal of Library & Information Science (IJLIS)*, 9(1), 3.
- Lan, C., Mohammed, K., Elly, B., & Happiness, D. (2025). The Role of Natural Language Processing (NLP) in Identifying Fraudulent Activities in Financial Communication and Documentation.
- Mehta, A., Nair, P., Kaya, O., & Singh, V. (2025). Financial fraud detection through K-means clustering-based unsupervised learning. *Economic and Financial Research Letters*, 5(5), 1–XX.
- Na, G., Park, M., Cha, H., & Chai, S. (2025). Human-centered LLM-agent system for detecting anomalous digital asset transactions.
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2024). A comprehensive overview of large language models. *arXiv*.
- Nawaiseh, A. K., Abbod, M. F., & Itagaki, T. (2020). Financial statement audit using support vector machines, artificial neural networks, and K-nearest neighbor: An empirical study of UK and Ireland. *International Journal of Simulation Systems, Science & Technology*, 21(2), 1–6.
- Nieschwietz, R. J., Schultz Jr, J. J., & Zimbelman, M. F. (2000). Empirical research on external auditors. *Detection of Financial Statement Fraud*, *Journal of Accounting Literature*, 19, 190-246.

- Offermann, Philipp & Blom, Sören & Schönherr, Marten & Bub, Udo. (2010). Artifact Types in Information Systems Design Science – A Literature Review. 77-92. 10.1007/978-3-642-13335-0_6.
- Offermann, Philipp & Levina, Olga & Schönherr, Marten & Bub, Udo. (2009). Outline of a design science research process. 10.1145/1555619.1555629.
- Onyenahazi, O. B. (2025). Integrating artificial intelligence into financial auditing to enhance accuracy, efficiency, and regulatory compliance outcomes. *International Journal of Advance Research Publication and Reviews*, 2(7), 23–44. ISSN 3049-0103
- Peffers, Ken & Tuunanen, Tuure & Rothenberger, Marcus & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*. 24. 45-77.
- Power, M. (2000). The audit society—Second thoughts. *International Journal of Auditing*, 4(1), 111-119.
- Ramamoorti, S. (2003). Internal auditing: history, evolution, and prospects.
- Relacion, P. J. Qualitative Research Methods DEFINITION OF QUALITATIVE RESEARCH.
- Schultz, M., & Tropmann-Frick, M. (2020). Autoencoder Neural Networks versus External Auditors: Detecting Unusual Journal Entries in Financial Statement Audits. *Hawaii International Conference on System Sciences*.
- Sharma, A., & Panigrahi, P. K. (2013). A review of financial accounting fraud detection based on data mining techniques. arXiv preprint arXiv:1309.3944.
- Sheikh, H., Prins, C., & Schrijvers, E. (2023). AI as a system technology. In *Mission AI: The New System Technology* (pp. 85-134). Cham: Springer International Publishing.
- Skulmoski, G. J., Hartman, F. T., & Krahm, J. (2007). The Delphi method for graduate research. *Journal of Information Technology Education: Research*, 6(1), 1-21
- Smith, G. S., & Crumbley, D. L. (2009). Defining a forensic audit. *Journal of Digital Forensics, Security and Law*, 4(1), 3.
- Watson, A., Richards, G., & Schiff, D. (2025). Explain first, trust later: LLM-augmented explanations for graph-based crypto anomaly detection.
- Wijaya, J. R. T., Prasetyo, I., Rahmatika, D. N., & Indriasih, D. (2025). ARTIFICIAL INTELLIGENCE AND AUDIT QUALITY: AN EMPIRICAL LITERATURE REVIEW FROM SCOPUS DATABASE. *Fokus Ekonomi: Jurnal Ilmiah Ekonomi*, 20(1), 61-76.

Wirth, R., & Hipp, J. (2000, April). CRISP-DM: Towards a standard process model for data mining. In Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining (Vol. 1, pp. 29-39).

Zhao, Y. (2025). Challenges and limitations of traditional financial auditing in the big data environment. *International Journal of Finance and Investment*, 3(2), 1–11

Statement of Autonomy

I hereby declare that this thesis was written independently by George Samy Nabih, using only the stated sources and aids. All parts taken from external sources, whether directly or indirectly, have been clearly marked as such.

George Samy Nabih

May-2026